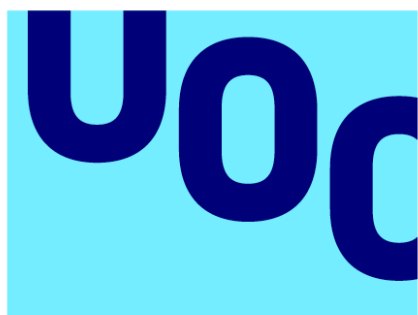


# Detección automática de *phishing* mediante *Machine Learning*



Universitat  
Oberta  
de Catalunya

---

**Sergio Cano San José**

Grado Ingeniería Informática  
Seguridad Informática

**Nombre Tutor/a de TF**

Jorge Miguel Moneo

**Profesor/a responsable de  
la asignatura**

Jorge Miguel Moneo

**Fecha Entrega: 28/05/2026**



Esta obra está sujeta a una licencia de Reconocimiento-NoComercial-SinObraDerivada [3.0 España de Creative Commons](https://creativecommons.org/licenses/by-nc-nd/3.0/es/)

## FICHA DEL TRABAJO FINAL

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Título del trabajo:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <i>Detección automática de phishing mediante Machine Learning</i> |
| <b>Nombre del autor:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <i>Sergio Cano San José</i>                                       |
| <b>Nombre del director/a:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <i>Jorge Miguel Moneo</i>                                         |
| <b>Nombre del PRA:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <i>Jorge Miguel Moneo</i>                                         |
| <b>Fecha de entrega (mm/aaaa):</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | <i>05/2026</i>                                                    |
| <b>Titulación o programa:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Grado de Ingeniería Informática                                   |
| <b>Área del Trabajo Final:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <i>Seguridad Informática</i>                                      |
| <b>Idioma del trabajo:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <i>Castellano</i>                                                 |
| <b>Palabras clave</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <i>Phishing - Machine Learning - Detección</i>                    |
| <b>Resumen del Trabajo</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                   |
| <p>El <i>phishing</i> constituye una de las amenazas más frecuentes y efectivas en el ámbito de la ciberseguridad, afectando tanto a usuarios individuales como a organizaciones. Los métodos tradicionales de detección, basados en listas negras o reglas estáticas, presentan limitaciones frente a ataques cada vez más dinámicos, automatizados y técnicamente sofisticados.</p> <p>Este trabajo desarrolla y evalúa un sistema de detección automática de <i>phishing</i> basado en <i>Machine Learning</i>, orientado a clasificar URLs y características asociadas a páginas web como legítimas o fraudulentas. Para ello, se implementa un motor híbrido de extracción de 25 atributos, combinando análisis léxico de la URL con indicadores estáticos extraídos del contenido HTML. La metodología emplea <i>datasets</i> públicos de distintas etapas temporales, incluyendo un conjunto antiguo de 2018, el <i>dataset</i> PhiUSIIL de 2024 y una validación externa con muestras activas obtenidas en 2026 mediante PhishTank y dominios legítimos de alta reputación.</p> <p>El estudio compara el rendimiento de Random Forest, Support Vector Machines y XGBoost bajo condiciones homogéneas de entrenamiento y validación. Los resultados permiten analizar la obsolescencia de los patrones históricos de <i>phishing</i>, evidenciando el impacto de la deriva de datos sobre la capacidad de generalización de los modelos. Finalmente, se identifican las limitaciones prácticas del sistema, se evalúa el equilibrio entre Falsos Positivos y Falsos Negativos, y se proponen líneas de mejora orientadas a incrementar la robustez, eficiencia y aplicabilidad del modelo en entornos reales.</p> |                                                                   |

## Abstract

Phishing remains one of the most frequent and persistent threats in cybersecurity, affecting both individual users and organizations. Traditional detection mechanisms based on blacklists or static rules show clear limitations when facing increasingly dynamic, automated and technically sophisticated attack campaigns.

This work develops and evaluates an automatic phishing detection system based on Machine Learning, aimed at classifying URLs and website-related features as legitimate or fraudulent. To achieve this, a hybrid feature extraction engine composed of 25 attributes is implemented, combining lexical URL analysis with static indicators extracted from HTML content. The methodology relies on public datasets from different time periods, including an older 2018 dataset, the 2024 PhiUSIIL dataset, and an external validation set composed of active phishing samples collected in 2026 through PhishTank and high-reputation legitimate domains.

The study compares the performance of Random Forest, Support Vector Machines and XGBoost under homogeneous training and validation conditions. The results make it possible to analyze the obsolescence of historical phishing patterns, highlighting the impact of data drift on the generalization capacity of the models. Finally, the work identifies the practical limitations of the system, evaluates the trade-off between False Positives and False Negatives, and proposes improvement lines aimed at increasing the robustness, efficiency and real-world applicability of the model.

# ÍNDICE

|       |                                                                           |    |
|-------|---------------------------------------------------------------------------|----|
| 1     | Introducción.....                                                         | 8  |
| 1.1   | Contexto y justificación del Trabajo .....                                | 8  |
| 1.2   | Objetivos del Trabajo.....                                                | 9  |
| 1.3   | Impacto en sostenibilidad, ético-social y de diversidad .....             | 9  |
| 1.4   | Enfoque y método seguido .....                                            | 10 |
| 1.5   | Planificación del Trabajo.....                                            | 11 |
| 1.6   | Breve resumen de productos obtenidos .....                                | 12 |
| 1.7   | Breve descripción de los otros capítulos de la memoria.....               | 13 |
| 1.8   | Declaración sobre el uso de inteligencia artificial generativa.....       | 14 |
| 2     | Estado del Arte.....                                                      | 15 |
| 2.1   | El fenómeno del <i>phishing</i> y su evolución.....                       | 15 |
| 2.1.1 | Evolución técnica y sofisticación de los vectores de ataque.....          | 17 |
| 2.2   | Fundamentos de <i>Machine Learning</i> en Ciberseguridad .....            | 18 |
| 2.3   | Arquitecturas algorítmicas para la clasificación .....                    | 19 |
| 2.3.1 | Random Forest.....                                                        | 20 |
| 2.3.2 | Support Vector Machines (SVM).....                                        | 20 |
| 2.3.3 | XGBoost (eXtreme Gradient Boosting).....                                  | 21 |
| 2.4   | Detección de <i>phishing</i> mediante <i>Machine Learning</i> .....       | 21 |
| 2.4.1 | Enfoques basados en el análisis léxico .....                              | 21 |
| 2.4.2 | Enfoques basados en contenido .....                                       | 22 |
| 2.5   | Análisis de soluciones actuales y brecha tecnológica .....                | 22 |
| 3     | Materiales y métodos .....                                                | 24 |
| 3.1   | Análisis de requisitos del sistema .....                                  | 24 |
| 3.1.1 | Requisitos funcionales.....                                               | 24 |
| 3.1.2 | Requisitos no funcionales .....                                           | 24 |
| 3.2   | Estudio de obsolescencia de los conjuntos de datos de entrenamiento ..... | 25 |
| 3.2.1 | Selección de fuentes de datos ( <i>Datasets</i> ).....                    | 25 |
| 3.2.2 | Configuración del muestreo y distribución de datos.....                   | 26 |
| 3.2.3 | El fenómeno de la deriva de datos ( <i>Data Drift</i> ) .....             | 27 |
| 3.3   | Evaluación de algoritmos de aprendizaje .....                             | 27 |
| 3.3.1 | Selección de modelos candidatos .....                                     | 27 |
| 3.3.2 | Configuración del muestreo y distribución de datos.....                   | 29 |
| 3.4   | Arquitectura lógica y diseño experimental .....                           | 30 |
| 3.4.1 | Protocolo de entrenamiento y validación .....                             | 30 |
| 3.4.2 | Protocolo de experimentación y configuración de datos.....                | 31 |
| 3.5   | Diseño y taxonomía de atributos del motor de extracción .....             | 32 |
| 3.5.1 | Atributos de análisis léxico (Nivel 1) .....                              | 32 |
| 3.5.2 | Atributos de contenido HTML (Nivel 2).....                                | 33 |
| 3.5.3 | Justificación de la selección de atributos.....                           | 34 |
| 3.5.4 | Validación experimental del motor de extracción.....                      | 34 |

|       |                                                                               |    |
|-------|-------------------------------------------------------------------------------|----|
| 3.6   | Métricas de evaluación y criterios de rendimiento .....                       | 36 |
|       | Definición de Tipos de Error y Seguridad Crítica .....                        | 36 |
|       | Indicadores de Rendimiento .....                                              | 37 |
| 3.7   | Validación del código generado mediante IA.....                               | 38 |
| 3.8   | Implementación y validación del modelo final.....                             | 38 |
| 3.8.1 | Niveles de validación del sistema .....                                       | 38 |
| 3.8.2 | Criterios de compromiso para la selección del modelo óptimo.....              | 39 |
| 4     | Resultados .....                                                              | 40 |
| 4.1   | Análisis de obsolescencia de los patrones de detección .....                  | 40 |
| 4.1.1 | Rendimiento y evaluación del modelo antiguo ( <i>Dataset</i> 2018).....       | 40 |
| 4.1.2 | Rendimiento y evaluación del modelo contemporáneo ( <i>Dataset</i> 2024)..... | 42 |
| 4.1.3 | Diagnóstico comparativo de la brecha generacional.....                        | 44 |
| 4.2   | Análisis de algoritmos.....                                                   | 48 |
| 4.2.1 | Verificación de convergencia de los modelos (Validación interna) .....        | 49 |
| 4.2.2 | Evaluación comparativa de modelos (Validación Externa 2026) .....             | 49 |
| 4.2.3 | Análisis de viabilidad y operativa.....                                       | 52 |
| 5     | Conclusiones y trabajos futuros .....                                         | 54 |
| 5.1   | Evaluación del cumplimiento de objetivos .....                                | 54 |
| 5.2   | Análisis del seguimiento de la planificación.....                             | 56 |
| 5.3   | Evaluación del impacto ético, social y de sostenibilidad.....                 | 57 |
| 5.4   | Conclusiones del Estudio de Obsolescencia .....                               | 59 |
| 5.5   | Conclusiones del Análisis de Algoritmos .....                                 | 61 |
| 5.6   | Análisis de Limitaciones .....                                                | 62 |
| 5.7   | Propuestas de mejora y robustez del sistema .....                             | 65 |
| 5.8   | Líneas de trabajo futuro.....                                                 | 67 |
| 6     | Glosario.....                                                                 | 69 |
| 7     | Bibliografía .....                                                            | 73 |
| 8     | Anexos .....                                                                  | 76 |

## LISTA DE FIGURAS

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| <b>Figura 1:</b> Diagrama de Gantt detallado del proyecto (Semanas 1-13) .....               | 12 |
| <b>Figura 2:</b> Evolución anual de sitios de <i>phishing</i> únicos (2016-2025).....        | 16 |
| <b>Figura 3:</b> Comparativa de soluciones de seguridad (Forrester Wave 2025)....            | 23 |
| <b>Figura 4:</b> Arquitectura lógica del <i>pipeline</i> de detección y flujo de datos ..... | 30 |
| <b>Figura 5:</b> Curva ROC combinada de los modelos de 2018 y 2024.....                      | 44 |
| <b>Figura 6:</b> Comparativa de importancia de atributos (2018 vs 2024).....                 | 48 |
| <b>Figura 7:</b> Comparativa de Curvas ROC: RF vs SVM vs XGBoost.....                        | 51 |

## LISTA DE TABLAS

|                                                                                                                                           |    |
|-------------------------------------------------------------------------------------------------------------------------------------------|----|
| <b>Tabla 1:</b> Comparativa de algoritmos de clasificación .....                                                                          | 28 |
| <b>Tabla 2:</b> Comparativa de rendimiento y fidelidad entre la representación de datos original y la generada por el motor híbrido ..... | 35 |
| <b>Tabla 3:</b> Sensibilidad del modelo 2018 ante muestras de 2026.....                                                                   | 41 |
| <b>Tabla 4:</b> Métricas globales (Umbral 0.90) – Modelo 2018.....                                                                        | 41 |
| <b>Tabla 5:</b> Matriz de confusión (Modelo 2018 / Umbral 0.90) .....                                                                     | 41 |
| <b>Tabla 6:</b> Sensibilidad del modelo 2024 ante muestras de 2026.....                                                                   | 42 |
| <b>Tabla 7:</b> Métricas globales (Umbral 0.20) – Modelo 2024.....                                                                        | 43 |
| <b>Tabla 8:</b> Matriz de confusión (Modelo 2024 / Umbral 0.20) .....                                                                     | 43 |
| <b>Tabla 9:</b> Comparativa de rendimiento entre el modelo 2018 y el modelo 2024 ( <i>Test</i> 2026).....                                 | 44 |
| <b>Tabla 10:</b> Matriz comparativa de evolución de atributos en muestras de <i>phishing</i> (2018 vs 2024) .....                         | 46 |
| <b>Tabla 11:</b> Sensibilidad comparativa ante muestras de 2026.....                                                                      | 50 |
| <b>Tabla 12:</b> Métricas detalladas por arquitectura (Umbrales óptimos).....                                                             | 50 |
| <b>Tabla 13:</b> Matriz de confusión (SVM: Umbral 0.25) .....                                                                             | 50 |
| <b>Tabla 14:</b> Evaluación de la eficiencia operativa .....                                                                              | 52 |

# 1 Introducción

## 1.1 Contexto y justificación del Trabajo

El *phishing* se define como un modelo de ciberataque basado en técnicas de ingeniería social donde los atacantes utilizan correos electrónicos, mensajes de texto o sitios web fraudulentos para engañar a los usuarios y obtener datos confidenciales [1]. Esta práctica se ha consolidado como una de las principales amenazas en el ecosistema de la ciberseguridad actual, actuando como el vector de entrada en la gran mayoría de los incidentes de robo de identidad y brechas de datos a nivel global. Según el informe de la APWG [2], el volumen de ataques alcanzó un máximo histórico de 5 millones de detecciones anuales en 2023, estabilizándose desde entonces en cifras que superan los 3.7 millones de sitios únicos detectados al año. Esta escala industrial de fraude sugiere que los mecanismos tradicionales, basados en listas negras y reglas estáticas, son insuficientes para gestionar amenazas que mutan y se automatizan en tiempo real.

La aplicación de técnicas de *Machine Learning* permite identificar patrones no lineales en los datos que pueden pasar desapercibidos para los sistemas convencionales. No obstante, la rápida evolución de las técnicas de ingeniería social genera una brecha tecnológica relevante debido a la integración de la Inteligencia Artificial Generativa. Esta tecnología permite a los atacantes automatizar la creación de señuelos hiperrealistas, eliminando errores gramaticales y adaptando el lenguaje de forma personalizada, lo que reduce drásticamente el tiempo de creación de una campaña de varias horas a apenas unos minutos [1]. Esta capacidad de escalado industrial de amenazas optimizadas mediante IA justifica la necesidad de investigar modelos que no solo sean precisos en entornos de laboratorio, sino que demuestren robustez y capacidad de generalización ante la sofisticación de los vectores de ataque contemporáneos.

La eficacia de estos sistemas puede verse condicionada por la vigencia de los *datasets* empleados, ya que la posible diferencia entre patrones antiguos y amenazas actuales sugiere la necesidad de evaluar la obsolescencia de los datos. Para abordar esta variabilidad, el proyecto propone un motor de extracción híbrido que combine el análisis léxico de la URL con la inspección del contenido HTML. Esta aproximación busca integrar tanto la estructura de la dirección web como los atributos técnicos de la página, tratando de ofrecer una detección más robusta frente a las infraestructuras de fraude detectadas en la actualidad.

### Aportación e Innovación del Proyecto

La aportación principal de este trabajo reside en el ciclo completo de desarrollo y evaluación crítica de un sistema de detección de *phishing* con el objetivo de aproximarse a su aplicación en entornos reales. A diferencia de enfoques puramente teóricos, este proyecto aporta valor mediante un análisis comparativo a diferentes niveles: primero, evaluando la obsolescencia de los patrones de ataque a través de diferentes fuentes de datos y, segundo, analizando el rendimiento de diversos algoritmos para identificar la arquitectura más eficiente.



La innovación se centra en el estudio de las limitaciones prácticas del sistema obtenido. El trabajo no se limita a presentar un modelo con alta precisión, sino que dedica una fase fundamental a la validación con muestras activas para detectar fallos de robustez. De este modo, la contribución final del proyecto consiste en la formulación de mejoras incrementales basadas en datos experimentales, proporcionando posibles líneas de mejora técnica para optimizar la aplicabilidad de los sistemas de *Machine Learning* en la defensa contra el fraude digital contemporáneo.

## 1.2 Objetivos del Trabajo

El objetivo general del trabajo es diseñar e implementar un sistema propio basado en arquitecturas de *Machine Learning* para la detección automática de *phishing* en URLs, así como analizar sus limitaciones y proponer posibles mejoras orientadas a incrementar su robustez y aplicabilidad en entornos reales.

De este objetivo general se derivan los siguientes objetivos específicos:

- Seleccionar y caracterizar conjuntos de datos de diversas fuentes para establecer una base de entrenamiento que cubra tanto patrones clásicos como amenazas modernas.
- Diseñar e implementar un motor de extracción de características híbrido capaz de procesar atributos léxicos de la URL y atributos de contenido HTML en tiempo real.
- Realizar el preprocesamiento y normalización semántica de los datos para asegurar interoperabilidad entre diferentes fuentes y arquitecturas de modelos.
- Implementar y comparar distintos algoritmos de clasificación supervisada para identificar la arquitectura que mejor generalice ante URLs no vistas previamente.
- Evaluar el rendimiento de los modelos mediante métricas de clasificación (como precisión y análisis de Falsos Positivos/ Falsos Negativos) utilizando muestras activas obtenidas en PhishTank.
- Analizar las limitaciones prácticas de los modelos obtenidos.
- Formular y validar experimentalmente posibles mejoras orientadas a aumentar la robustez del sistema.

## 1.3 Impacto en sostenibilidad, ético-social y de diversidad

El desarrollo de este proyecto se ha diseñado bajo criterios de responsabilidad técnica, evaluando las implicaciones del uso de *Machine Learning* en la detección de amenazas.

### Consideraciones Éticas y Privacidad

El sistema opera exclusivamente sobre identificadores públicos (URLs) y estructuras de código HTML accesibles en la red, orientando el diseño a evitar la captura o el procesamiento de información de carácter personal. Siguiendo las

directrices de la AEPD [3] en su guía sobre “10 Malentendidos sobre el *Machine Learning*”, el proyecto emplea un volumen de unas 230.000 instancias con el fin de reforzar la representatividad estadística del modelo, buscando un equilibrio entre la precisión de las muestras y la acumulación masiva de registros. Desde el punto de vista social, el proyecto busca ofrecer una herramienta técnica para mitigar el fraude digital, especialmente relevante ante la sofisticación de los ataques impulsada por la Inteligencia Artificial [4]. Por otro lado, existe el riesgo de que este tipo de sistemas genere una excesiva dependencia en mecanismos automatizados de decisión, lo que podría reducir la capacidad crítica del usuario final frente a ataques de ingeniería social.

### **Sostenibilidad Computacional**

A diferencia de los modelos de aprendizaje profundo (*Deep Learning*) que requieren *hardware* de alto rendimiento y gran consumo energético, este estudio prioriza algoritmos de aprendizaje supervisado como Random Forest, SVM o XGBoost. Esta elección técnica se alinea con el concepto de Green AI [5], el cual aboga por la eficiencia algorítmica como una métrica de éxito tan relevante como la precisión. De este modo, el proceso de entrenamiento puede ejecutarse en hardware convencional con un consumo energético reducido, lo que apunta a una solución eficiente y viable incluso en entornos con recursos limitados. Adicionalmente, este tipo de modelos podría ser analizado por actores maliciosos para diseñar ataques que evadan sus patrones de detección, lo que refuerza la necesidad de una actualización continua del sistema.

### **Análisis de Sesgo y Diversidad de Datos**

El proyecto aborda el sesgo temporal mediante un estudio de obsolescencia, identificando qué patrones de ataque del pasado ya no son válidos en el contexto actual. Para el modelo final, se prioriza el uso de un *dataset* contemporáneo con el fin de asegurar que el sistema sea efectivo contra amenazas actuales. Además, el análisis de falsos positivos es una fase crítica del trabajo, ya que se busca evitar el bloqueo injustificado de sitios legítimos que compartan características técnicas con páginas fraudulentas.

## **1.4 Enfoque y método seguido**

El trabajo presenta un enfoque experimental y comparativo, orientado al desarrollo de un sistema de clasificación de URLs robusto. Para la gestión del proyecto, se emplea una metodología *Agile*, basada en un desarrollo iterativo e incremental. Esto permite realizar ciclos rápidos de entrenamiento, evaluación y ajuste de los modelos, facilitando la incorporación de mejoras a medida que se analizan los resultados de los experimentos y la calidad de las características extraídas.

### **Metodología Experimental**

El estudio se desarrolla a través de una fase de experimentación controlada, utilizando *datasets* públicos etiquetados para el entrenamiento y ajuste de modelos, seguida de una fase de validación externa. Se ha implementado un flujo de trabajo que incluye el preprocesamiento de datos, la normalización semántica entre diferentes fuentes y la extracción de características mediante un

motor híbrido (léxico y HTML). La evaluación de los resultados no solo incluirá métricas de precisión estándar, sino que se centrará en la capacidad de generalización ante muestras nunca antes vistas por el modelo.

### **Entorno Tecnológico**

Para garantizar la reproducibilidad y escalabilidad del proyecto, el desarrollo se ha realizado en Python. Se ha empleado el ecosistema de librerías especializadas en *Machine Learning* y análisis de datos para el entrenamiento y la validación de algoritmos. Esta elección tecnológica asegura una arquitectura flexible que permite comparar diversas familias de algoritmos (como Random Forest o SVM) bajo las mismas condiciones experimentales.

### **Criterios de Evaluación y Análisis Crítico**

Más allá de la implementación algorítmica, el método seguido prioriza el análisis de robustez de los modelos. Esto implica:

- Identificar la degradación del rendimiento en función de la antigüedad de los patrones de ataque (estudio de obsolescencia).
- Evaluar el comportamiento del sistema ante URLs activas obtenidas de PhishTank.
- Realizar un diagnóstico de limitaciones prácticas, como el impacto de los falsos positivos en entornos reales para formular propuestas basadas en evidencias experimentales.

## **1.5 Planificación del Trabajo**

El desarrollo del trabajo se estructurará en fases secuenciales, donde cada etapa depende de la correcta finalización de la anterior.

Fase 1 – Definición y planificación (Semana 1):

- Definición del problema, objetivos, metodología y planificación general del proyecto.

Fase 2 – Revisión del estado del arte y selección de datos (Semanas 2-4):

- Revisión bibliográfica sobre técnicas de detección de *phishing* mediante *Machine Learning*.
- Selección del *dataset* y análisis inicial de los datos.
- Gestión de Riesgos: Ante la posibilidad de encontrar un *dataset* incompleto o desactualizado, se contempla como alternativa el uso de la combinación de varios *datasets* o el uso de repositorios alternativos para asegurar una base de entrenamiento sólida.

Fase 3 – Desarrollo y evaluación de modelos (Semanas 5-8):

- Preprocesamiento de datos y extracción de características.
- Implementación y entrenamiento de distintos modelos de clasificación.
- Evaluación preliminar y comparación de resultados.
- Gestión de Riesgos: Existe el riesgo de que la extracción de contenido HTML presente complicaciones técnicas elevadas o que las URLs de validación estén inactivas en el momento del análisis. Como medida de

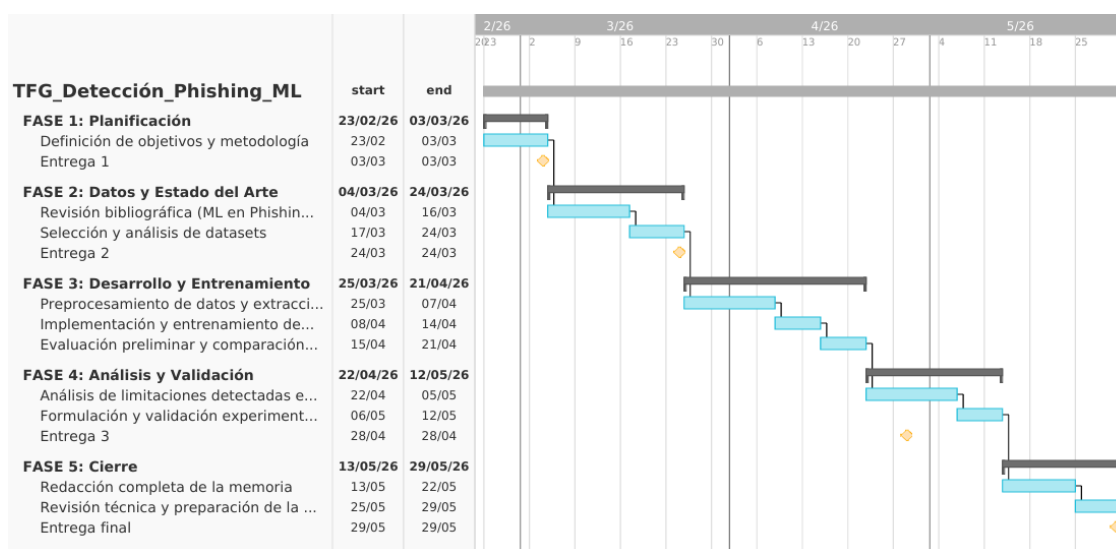
mitigación, en caso de inviabilidad o indisponibilidad, se realizará el análisis basándose exclusivamente en las características a nivel léxico de la URL, garantizando así la obtención de resultados y la viabilidad del estudio.

Fase 4 – Análisis crítico y propuestas de mejora (Semanas 9-11):

- Análisis de limitaciones detectadas en los modelos.
- Formulación y validación experimental de posibles mejoras orientadas a aumentar la robustez del sistema mediante el uso de muestras más actualizadas.

Fase 5 – Redacción y revisión final (Semanas 12-13):

- Redacción completa de la memoria.
- Revisión técnica y preparación de la entrega final.



**Figura 1:** Diagrama de Gantt detallado del proyecto (Semanas 1-13)

Fuente: Elaboración propia

## 1.6 Breve resumen de productos obtenidos

Como resultado del desarrollo del proyecto se han obtenido varios productos técnicos y experimentales que conforman la base del sistema propuesto. En primer lugar, se ha implementado un motor híbrido capaz de transformar URLs en un vector de 25 atributos, combinando atributos léxicos de la dirección web con indicadores estructurales extraídos del contenido HTML.

Se han generado y organizado distintos conjuntos de datos procesados, incluyendo muestras procedentes de un *dataset* antiguo (2018), datos contemporáneos de PhiUSIIL (2024) y muestras activas actuales obtenidas mediante PhishTank para la validación externa. Estos conjuntos han permitido realizar tanto el estudio de obsolescencia de patrones como la evaluación comparativa de algoritmos bajo condiciones equitativas.

También se han desarrollado notebooks experimentales en Python para el entrenamiento, validación y análisis de modelos de *Machine Learning*, incluyendo Random Forest, SVM y XGBoost. Estos *notebooks* recogen el flujo completo de preprocesamiento, entrenamiento, ajuste de umbrales, generación de métricas, matrices de confusión, curvas ROC y análisis de eficiencia computacional.

Finalmente, se ha elaborado un repositorio estructurado en GitHub que contiene los *notebooks* principales, los conjuntos de datos utilizados y los archivos procesados necesarios para documentar y reproducir los experimentos realizados. Este repositorio actúa como soporte técnico del trabajo, mientras que la presente memoria recoge el análisis metodológico, experimental y crítico de los resultados obtenidos.

## 1.7 Breve descripción de los otros capítulos de la memoria

Esta memoria se divide en los siguientes capítulos para detallar el proceso de desarrollo y experimentación del proyecto:

- **Capítulo 1 (Introducción):** Define el contexto del problema, la motivación del estudio, el alcance del proyecto y los objetivos generales y específicos que guían la investigación.
- **Capítulo 2 (Estado del Arte):** Se aborda el marco teórico del proyecto, analizando la evolución del *phishing* y los fundamentos del *Machine Learning* aplicados a la ciberseguridad. Además, se detallan las arquitecturas algorítmicas utilizadas para la clasificación (Random Forest, SVM y XGBoost) y se examinan los diferentes enfoques de detección documentados en las soluciones actuales.
- **Capítulo 3 (Materiales y Métodos):** Se detalla el diseño metodológico del estudio. En este bloque se describen los conjuntos de datos seleccionados, la implementación del motor híbrido de extracción de 25 características y el proceso de preprocesamiento y unificación semántica de variables. Del mismo modo, se formaliza la configuración de las familias algorítmicas evaluadas, las métricas de clasificación y el procedimiento de validación externa e interna.
- **Capítulo 4 (Resultados):** Constituye el núcleo técnico del trabajo. En primer lugar, se expone el estudio de obsolescencia cronológica al contrastar los modelos de 2018 y 2024 frente a la realidad de 2026, evaluando la deriva de datos y la reorganización de la importancia de las variables. Posteriormente, se presenta la comparativa algorítmica, analizando la capacidad de discriminación, el balance de errores y la eficiencia computacional de cada arquitectura.
- **Capítulo 5 (Conclusiones y Trabajos Futuros):** Recoge el cierre del proyecto mediante la evaluación del grado de cumplimiento de los objetivos y el análisis del impacto ético, social y de sostenibilidad. Finalmente, se sintetizan las limitaciones prácticas detectadas, las propuestas de mejora derivadas de los balances de error y las principales líneas de trabajo futuro planteadas.

## 1.8 Declaración sobre el uso de inteligencia artificial generativa

### Herramientas y Finalidad

Durante la Fase 3 (Desarrollo y evaluación de modelos), se han empleado de forma combinada las herramientas Gemini y ChatGPT como asistentes de apoyo técnico. El uso de estas tecnologías se ha centrado exclusivamente en la generación del código fuente necesario para la implementación del motor de extracción híbrido, los *scripts* de preprocesamiento y limpieza de datos, y la configuración de los algoritmos de clasificación supervisada (Random Forest, SVM y XGBoost). El proceso de implementación ha estado guiado por decisiones de diseño propias, utilizando la IA como herramienta de asistencia en la codificación bajo supervisión constante. Esta decisión técnica, consultada y validada previamente con el tutor del proyecto, se orientó a agilizar la fase de codificación para priorizar el análisis crítico de los resultados y el estudio de obsolescencia de los patrones de ataque.

### Metodología de Validación y Control

Dada la importancia crítica del sistema de detección, se ha llevado a cabo un protocolo de validación técnica con supervisión humana para cada bloque de código generado. Este proceso de control ha permitido identificar y corregir diversas inconsistencias producidas por la IA, entre las que destacan:

- Corrección de errores lógicos en el motor de extracción: Se detectaron fallos de precisión en las expresiones regulares y en la captura de atributos HTML que requerían de varios ajustes para asegurar la integridad de las variables.
- Corrección de errores de etiquetado: Se identificaron inversiones de polaridad en la asignación de clases (confusión entre etiquetas de *phishing* y legítimas) que habrían invalidado las métricas de rendimiento.
- Auditoría de coherencia: Se desarrollaron *scripts* de *test* adicionales para verificar que el flujo de datos entre los distintos módulos del sistema fuera coherente y estadísticamente representativo. Tal y como se detalla en el apartado 3.7, la ejecución de estos desarrollos resultó determinante para diagnosticar y subsanar los fallos lógicos mencionados.

### Valoración Crítica e Impacto

El impacto de la IA en este proyecto se valora como un elemento de apoyo que aumenta la eficiencia en la implementación técnica, permitiendo abordar un volumen de programación complejo en un tiempo reducido. A pesar de la agilidad aportada por estas herramientas, la validación final del contenido ha dependido de una revisión técnica profunda, indispensable para corregir las imprecisiones de la IA. Por ello, la responsabilidad final sobre la validez técnica y científica del sistema desarrollado recae íntegramente en el autor.

Se aclara explícitamente que la concepción de la metodología, la selección de variables de estudio, la interpretación de los resultados experimentales y la redacción íntegra de esta memoria son autoría propia, limitándose el uso de la IA a la capa de ejecución de código bajo auditoría constante.

## 2 Estado del Arte

### 2.1 El fenómeno del *phishing* y su evolución

#### Definición y Naturaleza

El *phishing* se define fundamentalmente como una forma de ingeniería social, un conjunto de técnicas psicológicas orientadas a manipular el comportamiento humano, llevando a la víctima a realizar acciones no deseadas, como la descarga de *malware* o la revelación de información sensible. A diferencia de las vulnerabilidades de software tradicionales, el *phishing* se centra en explotar los sesgos cognitivos de la víctima, aprovechando las limitaciones propias del comportamiento humano, convirtiendo al usuario en el punto de entrada más accesible para el atacante [6].

La eficacia de estos ataques radica en el uso de disparadores psicológicos diseñados para anular el pensamiento analítico. Los atacantes crean mensajes capaces de generar emociones intensas, tales como el miedo a una sanción financiera, la curiosidad ante una oferta exclusiva o el respeto hacia una autoridad suplantada. Al crear un falso sentido de urgencia, los atacantes logran que la víctima pase por alto señales de advertencia técnicas, priorizando una respuesta impulsiva sobre un análisis crítico del mensaje recibido.

#### La Evolución del *Phishing* en la Última Década

Durante los últimos diez años, el *phishing* ha dejado de ser una amenaza en el envío masivo de comunicaciones genéricas para consolidarse como el principal vector de entrada en la cadena de ataque de la ciberdelincuencia. Este fenómeno ha evolucionado en paralelo a la digitalización de los servicios críticos, transformándose en una infraestructura de fraude muy rentable y fácil de ampliar. La transición hacia entornos de trabajo remotos y el despliegue de servicios en la nube han ampliado la superficie de exposición, permitiendo que los atacantes refinen sus estrategias para vulnerar no solo a usuarios individuales, sino también a estructuras corporativas complejas mediante el robo de identidades digitales.

En este contexto, el registro histórico de incidentes recopilado por la Anti-*Phishing* Working Group (APWG) [2] permite identificar un cambio de paradigma en la escala y frecuencia de los ataques. Esta evolución no sigue un crecimiento lineal, sino que se manifiesta en picos de actividad impulsados por el contexto sociotécnico global y por la disponibilidad de nuevas herramientas de automatización para los atacantes.



**Figura 2:** Evolución anual de sitios de *phishing* únicos (2016-2025)

Fuente: Elaboración propia a partir de datos de APWG [2].

La métrica empleada en la Figura 2 corresponde al número de sitios de *phishing* únicos detectados, una unidad de medida que contabiliza las URL base de los servidores de ataque reportados. Esta metodología permite normalizar los datos al ignorar las variantes de enlaces dinámicos o personalizados que un mismo servidor puede generar para evadir filtros. Por tanto, el indicador de sitios únicos ofrece la representación más fiel de la infraestructura real utilizada por los atacantes y de la variedad de campañas activas a nivel global.

Bajo este criterio, el análisis de la gráfica revela que, tras un período de relativa estabilidad entre 2017 y 2019, el número de sitios de *phishing* únicos experimentó un crecimiento del 92% en el año 2020. Este incremento repentino, vinculado directamente a la digitalización forzada durante la crisis sanitaria, no supuso un evento aislado, sino el inicio de una escalada sostenida. En 2023, la cifra alcanzó un máximo histórico cercano a los 5 millones de detecciones anuales. Este máximo estuvo fuertemente influido por la expansión de herramientas de Inteligencia Artificial Generativa y de servicios de *Phishing-as-a-Service (PaaS)*, que facilitaron a los atacantes la automatización de infraestructuras fraudulentas con un nivel de sofisticación y escala sin precedentes.

A partir de 2024, la tendencia muestra una consolidación en niveles de alta intensidad, con una media anual superior a los 3.7 millones de sitios detectados. Si bien se observa una ligera corrección tras el máximo histórico de 2023, estas cifras confirman que el *phishing* ha alcanzado una fase de estabilización industrial. La magnitud de los registros actuales demuestra que los vectores de ataque se han consolidado como una amenaza continua y que las medidas de detección convencionales, basadas en la revisión manual o en firmas estáticas, resultan insuficientes para gestionar el volumen de amenazas generado por



sistemas automatizados. Este escenario refuerza la necesidad de implementar soluciones proactivas de *Machine Learning* que operen con una latencia mínima y sean capaces de adaptarse a la escala masiva de la infraestructura de fraude contemporánea.

### **2.1.1 Evolución técnica y sofisticación de los vectores de ataque**

Paralelamente al crecimiento en el volumen de incidentes, el *phishing* ha experimentado una transformación en sus mecanismos de ejecución. Esta evolución técnica ha dado lugar a distintas formas de ataque que han permitido a los atacantes evadir los sistemas de seguridad tradicionales y mejorar sus tasas de éxito.

#### **De la Difusión Masiva al *Spear Phishing***

En sus etapas iniciales, el *phishing* operaba principalmente mediante campañas de difusión masiva, consistentes en mensajes impersonales con la esperanza de que un pequeño porcentaje de usuarios interactuara con el señuelo. Sin embargo, la evolución técnica ha derivado en el *Spear Phishing*, una modalidad de ataque dirigido y altamente personalizado. A diferencia del *phishing* masivo, estas comunicaciones se basan en una fase previa de reconocimiento para suplantar identidades con las que la víctima mantiene una relación de confianza o jerarquía, como proveedores o directivos [7].

La relevancia de esta transición reside en su efectividad: aunque el *Spear Phishing* representa apenas el 0,1% del volumen total de correos maliciosos, es responsable del 66% de las vulneraciones de datos a nivel global [7]. Esta desproporción revela cómo las técnicas actuales de ingeniería social logran evadir los filtros basados en patrones genéricos y provocar un impacto mucho mayor.

#### **Uso de Técnicas de Ofuscación y Ataques Homógrafos**

La sofisticación técnica actual incluye el uso de ataques homógrafos basados en nombres de dominio internacionales (IDNs). Esta metodología aprovecha el estándar *Punycode* para registrar dominios que emplean caracteres de alfabetos no latinos que son visualmente casi idénticos a los caracteres ASCII tradicionales. Esta técnica permite suplantar dominios de alto tráfico mediante la sustitución de caracteres (por ejemplo, intercambiando una “o” latina por una “o” griega), lo que anula la inspección visual de la URL como método de defensa y permite que un sitio fraudulento mantenga una apariencia legítima mientras dirige al usuario hacia una infraestructura controlada por el atacante [8].

#### **Abuso de Protocolos de Seguridad y Nuevas Técnicas de Entrega**

Un factor determinante en la percepción de seguridad del usuario ha sido la creciente presencia del protocolo HTTPS en infraestructuras fraudulentas. Según el último informe de la APWG, el *phishing* registró 3.8 millones de ataques únicos durante 2025. Esta magnitud se ve agravada por el uso de vectores que evaden los perímetros de seguridad convencionales, como el envío de códigos QR maliciosos o el uso de SMS, técnicas que, según la APWG [2], a menudo no son capturadas por el filtrado de correo tradicional.

Este escenario demuestra que los indicadores de seguridad técnica tradicionales, como el icono del “candado” en el navegador, han perdido su capacidad para distinguir con precisión entre sitios legítimos y maliciosos. Este escenario hace necesaria la transición hacia modelos de *Machine Learning* capaces de analizar patrones de comportamiento y metadatos en tiempo real para identificar amenazas que resultan invisibles para los sistemas de inspección estáticos clásicos [2].

## 2.2 Fundamentos de *Machine Learning* en Ciberseguridad

### Definición y Aprendizaje Supervisado

El *Machine Learning* (ML) se define como una subdisciplina de la Inteligencia Artificial que se centra en el desarrollo de algoritmos capaces de identificar patrones y tomar decisiones a partir de datos, sin necesidad de una programación explícita para cada tarea. A diferencia de los sistemas basados en reglas rígidas, el ML permite que el sistema aprenda relaciones complejas a partir de conjuntos de datos y genere predicciones mediante el ajuste automático de sus parámetros internos [9].

Dentro de este campo, el aprendizaje supervisado es el enfoque más relevante para la detección de amenazas. Este enfoque se basa en el entrenamiento del modelo mediante un conjunto de datos previamente etiquetados, donde cada entrada está asociada a una respuesta conocida. El objetivo primordial de este proceso es que el algoritmo aprenda una función de mapeo que relacione las variables de entrada con la etiqueta correspondiente, permitiendo así que, ante datos nuevos y no vistos anteriormente, el sistema sea capaz de predecir la salida correcta con un alto grado de confianza.

### El Proceso de Clasificación

En el ámbito de la ciberseguridad, y específicamente en la identificación de sitios fraudulentos, el aprendizaje supervisado se implementa frecuentemente como un problema de clasificación binaria. Este proceso consiste habitualmente en asignar cada instancia de entrada a una de dos categorías excluyentes entre sí: clase maligna (*phishing*), o clase benigna (legítima). Si bien existen arquitecturas orientadas a clasificaciones multiclase, la naturaleza de este estudio se centra en la discriminación efectiva entre estas dos clases fundamentales. El éxito de esta tarea depende de un flujo de trabajo estructurado que transforma datos brutos en decisiones de clasificación, compuesto por las siguientes fases:

1. Entrada de datos y preprocesamiento: Recepción del conjunto de datos (*dataset*) y depuración de la información para eliminar ruidos o valores nulos que puedan introducir sesgos en el modelo.
2. Extracción de características: Identificación de los atributos numéricos o categóricos que mejor describen el comportamiento de una URL, como la longitud del dominio, la presencia de caracteres especiales. Estos atributos se organizan en un vector de características, que sirve como la representación matemática de la instancia para el algoritmo.

3. Entrenamiento del modelo: Fase en la que el algoritmo analiza las características de los datos etiquetados para ajustar sus parámetros internos y aprender a diferenciar los patrones de ambas clases.
4. Predicción y evaluación: Aplicación del modelo ya entrenado sobre nuevos datos para obtener una clasificación y medir su rendimiento mediante métricas de precisión [9].

### **Ventajas Frente a los Sistemas Basados en Reglas**

La principal limitación de la lógica de detección convencional radica en su dependencia de reglas estáticas y firmas predefinidas. En un sistema basado en heurísticas fijas, un programador debe identificar y codificar manualmente cada indicador de fraude mediante estructuras condicionales (if/else). Este enfoque resulta poco eficaz debido al carácter cambiante del *phishing*, donde los atacantes modifican constantemente los vectores de ataque para evadir patrones conocidos.

Frente a esta rigidez, el *Machine Learning* ofrece una ventaja competitiva basada en la capacidad de generalización. El objetivo no es que el modelo memorice el *dataset*, sino que aprenda reglas que le permitan identificar ataques nunca antes vistos. Los modelos de ML no solo son capaces de procesar volúmenes masivos de datos que serían inmanejables para un analista humano, sino que también pueden detectar patrones no lineales y relaciones sutiles entre las características de una URL. Mientras que un sistema de reglas podría fallar ante una mínima variación en el nombre de un dominio, un modelo de ML bien entrenado puede identificar la anomalía basándose en el comportamiento histórico de ataques similares, ofreciendo un enfoque más proactivo y adaptable a nuevas variantes de amenazas.

## **2.3 Arquitecturas algorítmicas para la clasificación**

Los algoritmos de *Machine Learning* suelen organizarse según la forma en que aprenden a partir de los datos y el tipo de problema que buscan resolver. De forma general, se distinguen tres enfoques principales [10]:

- **Aprendizaje Supervisado:** Se orienta a predecir resultados mediante la relación entre variables independientes y una “verdad fundamental” representada mediante datos etiquetados.
- **Aprendizaje no Supervisado:** Este enfoque busca identificar patrones y correlaciones presentes en conjuntos de datos sin etiquetas.
- **Aprendizaje por Refuerzo:** Un proceso iterativo de prueba y error donde el modelo busca maximizar una recompensa dentro de un entorno dinámico.

Dado que la detección de *phishing* se plantea como un problema de clasificación basado en muestras previamente etiquetadas como legítimas o fraudulentas, este proyecto se centra en el aprendizaje supervisado. Bajo este enfoque, se han seleccionado las siguientes tres arquitecturas para evaluar su eficacia en la mitigación de amenazas:

### 2.3.1 Random Forest

Esta arquitectura se fundamenta en el ensamblado de múltiples árboles de decisión, donde cada unidad se entrena mediante el uso de variaciones aleatorias de conjuntos de datos (*bagging*) y el uso de vectores aleatorios de características para garantizar la diversidad del modelo [11].

Según el estudio de Breiman, creador del algoritmo, la principal ventaja de este enfoque es que el error de generalización del bosque converge a un límite a medida que aumenta el número de árboles, lo que le otorga al sistema robustez frente al ruido y previene el sobreajuste (*overfitting*). Este fenómeno, conocido como *overfitting*, ocurre cuando un algoritmo memoriza los detalles y el ruido del conjunto de entrenamiento hasta el punto de ser incapaz de realizar predicciones precisas sobre datos nuevos.

En el contexto de la ciberseguridad, esta capacidad de generalización es importante, ya que permite al algoritmo identificar patrones de fraude incluso cuando las muestras presentan variaciones menores respecto al conjunto de entrenamiento. Además, el modelo permite calcular la importancia relativa de las variables, lo que en este proyecto resulta clave para determinar qué atributos de una URL o del código HTML conservan su peso predictivo frente a aquellos indicadores que han perdido vigencia debido a la evolución de las técnicas del *phishing*.

### 2.3.2 Support Vector Machines (SVM)

Esta arquitectura se define como un algoritmo de aprendizaje supervisado diseñado para clasificar datos mediante la búsqueda de un hiperplano de separación que maximice la distancia entre diferentes categorías en un espacio n-dimensional [12].

Desarrollado originalmente por Vladimir Vapnik, el objetivo principal del modelo es encontrar la frontera de decisión que garantice el margen más amplio posible entre los puntos más cercanos de clases opuestas. Al maximizar este margen respecto al hiperplano, el algoritmo logra una alta capacidad de generalización, permitiendo clasificar correctamente nuevos datos que no fueron vistos durante el entrenamiento. Una de las características de las SVM es su capacidad para gestionar problemas que no son linealmente separables mediante la aplicación de transformaciones matemáticas en un espacio de mayor dimensión, técnica denominada como el “truco del *kernel*”.

En el ámbito de la ciberseguridad, esta versatilidad es útil para detectar relaciones no lineales que los sistemas de inspección convencionales ignoran. Esta capacidad permite identificar anomalías sutiles en los metadatos de campañas de *phishing* que, pese a imitar la apariencia visual de portales legítimos, operan sobre infraestructuras de fraude cuyas firmas técnicas difieren de los patrones de tráfico legítimo.

### 2.3.3 XGBoost (eXtreme Gradient Boosting)

Esta arquitectura consiste en una implementación avanzada del algoritmo de potenciación del gradiente (*gradient boosting*), diseñada para optimizar la velocidad y la eficiencia computacional en el entrenamiento de modelos supervisados. A diferencia de otros métodos de ensamblado, este sistema construye árboles de forma secuencial para corregir los errores de las versiones anteriores mediante el descenso de gradiente [13].

La metodología, desarrollada originalmente por Tianqi Chen, destaca por su capacidad para combinar múltiples “aprendices débiles” hasta consolidar un modelo de predicción de alta precisión. Una de sus mayores ventajas es la implementación de algoritmos con reconocimiento de caché y computación paralela, lo que facilita el procesamiento de grandes volúmenes de datos de forma eficiente.

En el ámbito de la ciberseguridad, esta capacidad de respuesta resulta relevante para gestionar la escala de las amenazas actuales con una latencia mínima. La arquitectura permite identificar patrones complejos en el comportamiento de las campañas de *phishing* de forma automática, lo que puede contribuir a mantener su precisión predictiva ante infraestructuras de fraude que se despliegan de forma masiva.

## 2.4 Detección de *phishing* mediante *Machine Learning*

La aplicación del *Machine Learning* a la seguridad web ha permitido pasar de una detección basada en reglas estáticas y la consulta de listas negras de dominios previamente reportados, al uso de modelos predictivos que analizan las características de los datos en tiempo real [14]. En el ámbito específico de la detección de *phishing*, estos sistemas suelen estructurarse en dos enfoques principales, el análisis léxico de la URL y el análisis del contenido interno de la página, incluyendo código HTML y *scripts*.

### 2.4.1 Enfoques basados en el análisis léxico

Este enfoque se centra en el estudio de la cadena de caracteres que compone la URL, tratando el problema de detección como una tarea de análisis de texto. La principal ventaja de esta metodología es su eficiencia y velocidad, ya que permite clasificar un sitio web como sospechoso sin necesidad de acceder a su servidor ni descargar su contenido, lo que evita la exposición a posibles cargas útiles maliciosas.

La efectividad de estos modelos reside en su capacidad para identificar anomalías mediante algoritmos como Support Vector Machines (SVM) o Random Forest, los cuales permiten procesar eficazmente atributos como la longitud total de la dirección, el número de subdominios y el uso de caracteres especiales destinados a la suplantación de identidad. No obstante, a pesar de su alta escalabilidad, el análisis léxico puede resultar ineficiente ante ataques de gran sofisticación técnica que emplean dominios visualmente idénticos a los

legítimos. Esta limitación hace necesario complementar el análisis de la URL con el estudio técnico del código fuente y los componentes dinámicos integrados en el sitio web.

#### 2.4.2 Enfoques basados en contenido

A diferencia del análisis léxico, este enfoque examina la estructura interna de la página web para identificar indicadores de fraude que no son visibles en la dirección URL. Estos modelos analizan el código HTML y el comportamiento de los *scripts* para detectar elementos sospechosos, como formularios que solicitan credenciales de forma inusual, campos de entrada ocultos o redirecciones automáticas. Al profundizar en la composición del sitio, es posible identificar ataques que utilizan dominios visualmente legítimos pero cuya estructura interna revela su naturaleza maliciosa. De este modo, esta metodología permite detectar anomalías en el comportamiento técnico de la página, revelando amenazas que el análisis superficial de la dirección URL ignoraría por completo.

Un ejemplo reciente de esta tendencia hacia el análisis de datos híbridos es el conjunto de datos PhiUSIIL, propuesto por Prasad y Chandra [15]. Este *dataset* se caracteriza por integrar tanto características propias de la URL como una amplia variedad de atributos extraídos del contenido, incluyendo el análisis de etiquetas HTML y funciones *JavaScript*. Al emplear muestras actualizadas y una mayor diversidad de variables técnicas, este enfoque busca superar la tasa de falsos positivos de los modelos tradicionales, demostrando que la combinación de múltiples vectores de información es actualmente una de las estrategias más eficaces para el entrenamiento de sistemas de detección de *phishing*.

### 2.5 Análisis de soluciones actuales y brecha tecnológica

En la actualidad, la detección de *phishing* ha pasado de ser un proceso reactivo a uno preventivo basado en arquitecturas de Inteligencia Artificial. Líderes del sector como Google, Microsoft y Cloudflare han desarrollado soluciones relevantes mediante el despliegue de modelos de *Machine Learning* que operan en diferentes capas del ataque.

- Google ha implementado en sus herramientas (como Gemini y *Safe Browsing*) un sistema de defensa en profundidad que incluye clasificadores de contenido y mecanismos de detección de URLs sospechosas. Además, la compañía señala que Gmail bloquea automáticamente más del 99.9% del spam, intentos de *phishing* y malware antes de llegar al usuario [16].
- Microsoft Defender, por su parte, se enfoca en la “defensa autónoma”, para combatir ataques que escalan a compromisos multidominio en pocos minutos. Su modelo utiliza la correlación continua de señales para desplazar las tareas de investigación rutinaria hacia agentes de IA, permitiendo la interrupción automatizada de ataques complejos y reduciendo la carga de trabajo manual para los analistas [17].



- Cloudflare destaca por su tecnología de “huella digital de detección” (EDF), que emplea modelos de inteligencia artificial y aprendizaje automático para analizar atributos como el sentimiento del mensaje y contextos sospechosos, logrando una precisión declarada del 99.99% en la detección de *phishing* [18].

Más allá de estas tres organizaciones, el mercado de la ciberseguridad cuenta con un ecosistema robusto de proveedores que compiten por integrar capacidades de IA cada vez más sofisticadas. La relevancia de estas soluciones no solo se mide por su capacidad técnica, sino por su posición estratégica y la capacidad de respuesta ante un entorno de amenazas en constante evolución.

Para contextualizar el peso de estas tecnologías en el mercado global, resulta útil analizar evaluaciones profesionales. Como se detalla en la Figura 3, el análisis de Forrester Wave [19] categoriza a los principales actores del mercado en función de la solidez de su oferta tecnológica y su visión estratégica, estableciendo una comparativa directa sobre qué proveedores están definiendo el estado de la técnica en la detección de amenazas avanzadas.



**Figura 3:** Comparativa de soluciones de seguridad (Forrester Wave 2025)

Fuente: Forrester Research [19]

A pesar de estos avances, persiste una brecha tecnológica crítica. Los atacantes han comenzado a utilizar técnicas de *Prompt Injection* y suplantación de identidad generada por IA que intentan evadir estos mismos modelos. La existencia de estas soluciones de élite no elimina el riesgo para organizaciones con menos recursos o infraestructuras heredadas, lo que refuerza la necesidad de investigar modelos de clasificación eficientes y accesibles que utilicen *datasets* modernos para democratizar la detección avanzada.

## 3 Materiales y métodos

### 3.1 Análisis de requisitos del sistema

Para garantizar la viabilidad técnica y la eficacia operativa del sistema de detección de *phishing*, se han consolidado una serie de requisitos que rigen tanto su capacidad de ejecución como su robustez en entornos de ciberseguridad.

#### 3.1.1 Requisitos funcionales

Los requisitos funcionales definen las capacidades operativas que el sistema ha integrado para cumplir con los objetivos del estudio:

- **Extracción Híbrida:** El sistema debe ser capaz de procesar URLs para generar vectores de datos numéricos. Esto incluye la capacidad de analizar tanto la estructura léxica de la dirección web como los indicadores de riesgo presentes en el código HTML.
- **Evaluación Comparativa:** El diseño debe permitir la ejecución concurrente de las tres arquitecturas seleccionadas (Random Forest, SVM y XGBoost). Este requisito es fundamental para contrastar el rendimiento de cada modelo bajo las mismas condiciones de entrada y determinar cuál es la más robusta ante los intentos de fraude.
- **Integración de Muestras Externas:** El sistema debe ser capaz de procesar conjuntos de URLs provenientes de fuentes externas (como PhishTank), permitiendo su análisis mediante el motor de extracción y su uso en la validación de modelos.
- **Evaluación mediante Modelos Supervisados:** El sistema debe ser capaz de generar predicciones sobre las muestras analizadas mediante modelos entrenados, permitiendo comparar la etiqueta real con la estimada para evaluar su rendimiento.

#### 3.1.2 Requisitos no funcionales

Estos requisitos definen las propiedades y restricciones bajo las cuales el sistema debe operar:

- **Modularidad y Escalabilidad:** El sistema está diseñado con una estructura desacoplada que separa la extracción de características de la fase de entrenamiento y *test*. Esta estructura permite actualizar los algoritmos (RF, SVM, XGBoost) o modificar la lógica de extracción sin necesidad de reconstruir el sistema completo.
- **Eficiencia en el Procesamiento Masivo:** Dado que la extracción híbrida es la fase más costosa computacionalmente, el sistema integra una lógica de concurrencia mediante hilos (multihilo) y un sistema de almacenamiento de caché persistente. Esto ayuda a que el procesamiento de miles de URLs no se tenga que repetir en múltiples ocasiones y permite reanudar tareas interrumpidas sin pérdida de datos.



- **Portabilidad y Ecosistema Open Source:** El desarrollo es compatible con el ecosistema Python 3.x. Se basa en librerías de estándar industrial como Pandas para la gestión de datos, Scikit-Learn para la inteligencia artificial, y BeautifulSoup4 para el análisis de contenido, asegurando la reproducibilidad del experimento en cualquier entorno sin costes de licencia.
- **Seguridad y Privacidad:** El sistema no tratará datos personales sensibles, se limitará al análisis técnico de estructuras de URLs y código HTML público.
- **Aislamiento y Seguridad:** El motor implementa una técnica de análisis estático pasivo mediante la librería requests, que descarga el código fuente en texto plano sin renderizarlo. Al evitar la ejecución de *scripts* o código JavaScript, se reduce significativamente el riesgo de ejecución de cargas maliciosas, permitiendo que la inspección mediante BeautifulSoup se realice de forma aislada. Este método garantiza la integridad del sistema anfitrión durante el estudio de sitios de *phishing* activos.

## 3.2 Estudio de obsolescencia de los conjuntos de datos de entrenamiento

El objetivo principal de esta fase es evaluar la validez de las características métricas utilizadas hace casi una década frente a las amenazas contemporáneas. Se busca identificar qué indicadores clásicos han perdido capacidad predictiva debido a la evolución de las técnicas maliciosas.

### 3.2.1 Selección de fuentes de datos (*Datasets*)

Para realizar esta comparativa generacional y el análisis de obsolescencia, se han seleccionado cuatro fuentes de información bajo criterios de representatividad temporal y accesibilidad de los datos en bruto. La elección de estas fuentes frente a otros repositorios disponibles se justifica por la necesidad de disponer de las URLs originales. La mayoría de los *datasets* públicos solo proporcionan vectores de características ya procesados, lo que impediría aplicar un motor de extracción uniforme para garantizar una comparativa equitativa. De este modo, las fuentes seleccionadas permiten cubrir los requisitos técnicos necesarios para el estudio mediante los siguientes referentes:

- **Referente Antiguo (*Dataset* 2018 – Repositorio público PDD [\[20\]](#)):** Se utiliza un conjunto de URLs (~70000 muestras) recopilado en torno a 2018 y disponible en un repositorio público abierto. Aunque este *dataset* no dispone de una documentación formal ni de una trazabilidad completa de su proceso de construcción, se incluye por su utilidad metodológica, ya que, a diferencia de otros *datasets* antiguos que solo proporcionan atributos preprocesados, este permite el acceso a las URLs originales. Esta característica resulta esencial para el estudio de obsolescencia, ya

que garantiza que la comparación entre *datasets* se realice bajo una misma representación de variables.

- **Referente Contemporáneo (PhiUSIL [15], 2024):** Constituye el núcleo del entrenamiento moderno. Con más de 235000 muestras, este *dataset* representa la complejidad del *phishing* moderno, incluyendo técnicas de ofuscación y estructuras HTML dinámicas que no existían en la década anterior.
- **Muestra de Validación de Amenazas (PhishTank [21]):** Se han recolectado de forma selectiva muestras de URLs detectadas y verificadas como *phishing* por la comunidad de PhishTank. Durante esta recolección, se filtraron las muestras escogidas para evitar una alta redundancia de URLs muy similares, tratando de priorizar la diversidad de estructuras frente al volumen masivo. Estas muestras, extraídas durante la primera quincena de abril de 2026, actúan como el conjunto de prueba crítico para medir la obsolescencia, comprobando si los modelos (el de 2018 y el de 2024) son capaces de identificar intentos de fraude actuales.
- **Muestra de Validación de Sitios Legítimos (Dominios Top 10M [22]):** Para equilibrar las pruebas de validación externa, se han extraído muestras de la lista de los 10 millones de dominios principales basada en datos abiertos de Common Crawl. Concretamente, se ha seleccionado un subconjunto de los dominios mejor posicionados, filtrando únicamente aquellas URLs accesibles y procesables por el motor de extracción. Esta fuente garantiza que los sitios marcados como “legítimos” sean dominios con alta reputación y tráfico real, permitiendo evaluar la tasa de falsos positivos del sistema.

La combinación de estas fuentes permite construir un entorno experimental que abarca tanto el comportamiento histórico del *phishing* como su evolución reciente, garantizando una evaluación comparativa bajo condiciones equitativas.

### 3.2.2 Configuración del muestreo y distribución de datos

En esta fase se detalla el volumen de datos resultante tras someter las fuentes originales al motor de extracción híbrido. Es importante destacar que el estado de actividad de las URLs de *phishing* actúa como “cuello de botella”, ya que solo se integran aquellas muestras cuyo contenido HTML fue capturado y procesado con éxito:

- **Dataset 2018 (Repositorio público PDD):** Aunque se partió de un volumen bruto superior (~35000 URLs por clase), la tasa de disponibilidad de los sitios de *phishing* redujo la muestra procesable a 13000 registros. Para garantizar el equilibrio del experimento, se seleccionó un número idéntico de muestras legítimas, conformando un conjunto final de 26000 URLs.
- **Dataset 2024 (PhiUSIL):** Con el fin de asegurar una comparativa simétrica con el *dataset* antiguo, se ha extraído un subconjunto de 13000 muestras de *phishing* y 13000 legítimas (26000 totales), permitiendo que las variaciones en el rendimiento se deban exclusivamente a la evolución de los patrones y no al volumen de datos.

- **Validación Externa (PhishTank / Top Domains):** Se seleccionó una muestra inicial de 1000 URLs de *phishing* recientes y 1000 dominios legítimos. Tras el filtrado del motor de extracción, la persistencia de las amenazas activas limitó el conjunto a ~750 muestras de *phishing*, ajustándose el grupo de URLs legítimas al mismo número para mantener la simetría (~1500 muestras totales).

Este diseño experimental permite aislar el efecto del *dataset* de entrenamiento sobre el rendimiento de los modelos, garantizando que las diferencias observadas se deban a la evolución de los patrones de *phishing* y no a factores externos.

### 3.2.3 El fenómeno de la deriva de datos (*Data Drift*)

En el ámbito del *Machine Learning*, la deriva de datos o *Data Drift* ocurre cuando la distribución subyacente de los datos de entrada experimenta cambios significativos respecto al escenario de entrenamiento original [23]. En el contexto del estudio, este fenómeno representa el proceso por el cual las propiedades estadísticas y los patrones de uso del *phishing* evolucionan con el tiempo. Los atacantes modifican constantemente sus tácticas para evadir los sistemas de detección, alterando así la naturaleza de los datos que el modelo debe procesar.

Como consecuencia de este desplazamiento, un modelo basado en patrones antiguos puede perder eficacia al enfrentarse a una base de datos cuyas características ya no coinciden con las asunciones iniciales del algoritmo. Este estudio aborda dicha problemática mediante una comparativa generacional, evaluando cómo el cambio en la distribución de los datos de entrada impacta directamente en la capacidad predictiva de los modelos seleccionados frente a las amenazas actuales.

## 3.3 Evaluación de algoritmos de aprendizaje

El objetivo de esta fase es identificar la arquitectura algorítmica que presente una mejor capacidad de generalización. Para ello, se ha realizado una comparativa entre tres familias de algoritmos con fundamentos matemáticos distintos, permitiendo evaluar cuál se adapta mejor a la naturaleza de los datos de *phishing*.

### 3.3.1 Selección de modelos candidatos

Para fundamentar la arquitectura del sistema, se ha realizado una revisión técnica de los algoritmos de clasificación más extendidos en el ámbito del aprendizaje supervisado. La siguiente tabla presenta una comparativa técnica basada en criterios de rendimiento y aplicabilidad (ventajas y limitaciones):

**Tabla 1:** Comparativa de algoritmos de clasificación

| Algoritmo                                          | Familia matemática          | Ventajas                                                                                  | Limitaciones                                                                                                      |
|----------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Naive Bayes <a href="#">[24]</a>                   | Probabilística              | Bajo coste computacional y eficiencia en grandes volúmenes de datos.                      | Asunción de independencia condicional entre todas las variables de entrada.                                       |
| Logistic Regression <a href="#">[25]</a>           | Modelos Lineales            | Modelo estadístico simple y fácilmente interpretable.                                     | Limitación para capturar fronteras de decisión no lineales sin transformación previa de las variables.            |
| K-Nearest Neighbors (KNN) <a href="#">[26]</a>     | Basada en Distancias        | Capacidad para adaptarse a distribuciones de datos complejas sin asunciones previas.      | Elevado consumo de memoria y sensibilidad a la escala de las características.                                     |
| Decision Trees <a href="#">[27]</a>                | Árboles de Decisión         | Alta interpretabilidad del modelo mediante reglas lógicas de decisión.                    | Alta inestabilidad ante pequeñas variaciones en el conjunto de entrenamiento.                                     |
| Multi-layer Perceptron (MLP) <a href="#">[28]</a>  | Redes Neuronales            | Capacidad de aproximación de funciones complejas mediante múltiples capas.                | Presenta una menor interpretabilidad y requiere un volumen significativo de datos para generalizar correctamente. |
| Random Forest <a href="#">[11]</a>                 | Ensamble ( <i>Bagging</i> ) | Reducción de la varianza mediante el promediado de múltiples estimadores.                 | Mayor complejidad estructural y requerimientos de almacenamiento del modelo.                                      |
| Support Vector Machines (SVM) <a href="#">[12]</a> | Basada en Márgenes          | Robustez en espacios de alta dimensionalidad mediante el uso de funciones <i>kernel</i> . | Elevado coste computacional durante el proceso de entrenamiento                                                   |

|              |                              |                                                                       |                                                                                      |
|--------------|------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|              |                              |                                                                       | con datos masivos.                                                                   |
| XGBoost [13] | Ensamble ( <i>Boosting</i> ) | Optimización del sesgo mediante el aprendizaje secuencial de errores. | Alta dependencia de un ajuste preciso de hiperparámetros para evitar el sobreajuste. |

Fuente: Elaboración propia a partir de las fuentes citadas en la tabla

### Justificación y Criterios de Selección

La transición de esta comparativa general a la fase experimental del proyecto se fundamenta en tres requisitos técnicos: modelado no lineal, robustez ante ruido y viabilidad en la fase de clasificación. Bajo estos criterios, se justifica el descarte y selección de los modelos según la siguiente lógica:

- **Modelos Descartados por Insuficiencia Técnica:** Tanto Naive Bayes [24] como Logistic Regression [25] presentan limitaciones estructurales para capturar la interdependencia compleja entre los atributos léxicos y el contenido HTML. Por su parte, los Decision Trees [27] simples se descartan debido a su alta varianza y sensibilidad ante pequeñas variaciones, lo que podría comprometer la estabilidad del sistema ante nuevas amenazas.
- **Modelos Descartados por Ineficiencia Operativa:** KNN, como método de estimación local no paramétrico, requiere calcular la vecindad de cada nueva instancia durante la fase de predicción, lo que implica un coste computacional elevado al no disponer de una función de decisión global previamente aprendida [26]. Por su parte, el modelo MLP [28] (Redes Neuronales) añade una mayor complejidad técnica en el proceso de entrenamiento y ajuste, aspectos que no se alinean con la necesidad de simplicidad e interpretabilidad del presente estudio.
- **Modelos Seleccionados (Random Forest, SVM y XGBoost):** Estas tres arquitecturas se eligen por ofrecer un equilibrio óptimo entre los criterios definidos. Permiten evaluar el rendimiento del sistema desde tres enfoques matemáticos complementarios: la reducción de varianza (*Bagging*), la optimización de márgenes (SVM) y el aprendizaje iterativo de errores (*Boosting*).

Este conjunto de modelos asegura que el análisis comparativo sea representativo de las soluciones más solventes en el estado del arte de la ciberseguridad actual.

### 3.3.2 Configuración del muestreo y distribución de datos

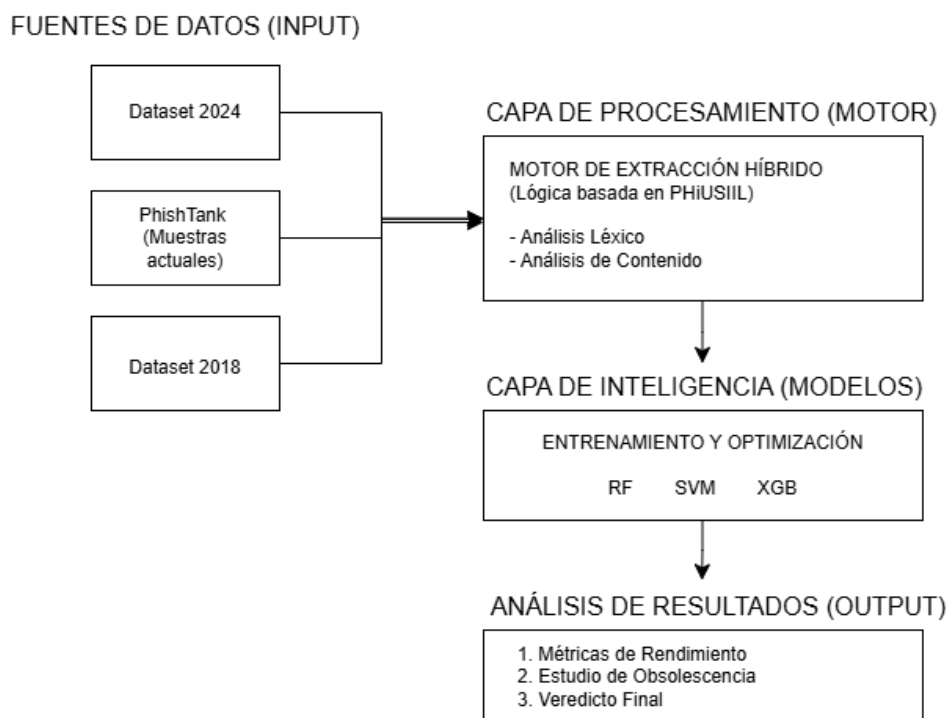
La distribución de los datos se ha diseñado para maximizar el aprendizaje de los modelos contemporáneos, manteniendo siempre el rigor del balanceo de clases:

- **Ajuste por disponibilidad en PhiUSIIL:** Aunque el *dataset* original cuenta con más de 235000 registros, el volumen final de entrenamiento se ve condicionado por la disponibilidad de las variables de contenido en las muestras de *phishing*. Se ha usado finalmente un conjunto de 50000 muestras por clase (100000 registros totales) para el entrenamiento de los modelos de 2024.
- **Validación Externa (PhishTank / Top Domains):** Se utiliza el mismo conjunto de ~1500 muestras (~750 *phishing*/ ~750 legítimas) detallado en el estudio de obsolescencia (3.2.2). Su aplicación es común para todos los modelos, garantizando una comparativa bajo un escenario de examen idéntico que permite medir la eficacia real de cada arquitectura frente a las mismas amenazas.

### 3.4 Arquitectura lógica y diseño experimental

#### 3.4.1 Protocolo de entrenamiento y validación

El diseño del sistema se ha articulado mediante una arquitectura modular que favorece la integridad de los datos a lo largo de todo el ciclo de procesamiento. Esta estructura permite separar claramente la ingesta de información, la generación de variables y la fase de clasificación, de forma que el flujo de procesamiento sea idéntico para cada una de las fuentes analizadas:



**Figura 4:** Arquitectura lógica del *pipeline* de detección y flujo de datos

Fuente: Elaboración propia mediante la herramienta Draw.io

Como se observa en la Figura 4, el diseño se fundamenta en un proceso de extracción estructurado y coherente, organizado en cuatro capas:

1. **Capa de Fuentes (Input):** Se integran tres entornos de datos con naturalezas temporales distintas: el referente antiguo (*Dataset* 2018), el referente contemporáneo (*Dataset* 2024) y el conjunto de muestras activas recopiladas a partir de PhishTank. Esta variedad de entradas es la que permite analizar la evolución de los patrones de ataque en distintos contextos temporales.
2. **Capa de procesamiento (Motor):** Es el núcleo técnico del proyecto. El motor de extracción híbrido aplica una lógica de captura doble (Léxica y de Contenido). Su diseño está estrictamente alineado con la metodología de PhiUSIIL, lo que favorece la compatibilidad de los datos extraídos de URLs actuales con los modelos entrenados.
3. **Capa de Inteligencia (Modelos):** Los vectores de atributos resultantes alimentan simultáneamente a las tres familias de algoritmos seleccionadas (RF, SVM y XGBoost). Es aquí donde se ejecutan los procesos de entrenamiento y ajuste de parámetros, permitiendo que cada modelo aprenda los patrones de fraude de forma independiente para su posterior comparativa.
4. **Capa de Resultados (Output):** El flujo concluye con una fase de análisis. En primer lugar, se evalúa el rendimiento técnico de los algoritmos mediante métricas estándar definidas más adelante, identificando la arquitectura más eficiente. Complementariamente, se realiza un estudio de obsolescencia para determinar la vigencia de los patrones de ataque, y finalmente, se emite una evaluación final sobre la robustez global del sistema frente a las amenazas actuales.

Este enfoque modular permite que el sistema sea escalable y que los resultados obtenidos sean fruto de un proceso de ingeniería de datos coherente y reproducible.

### 3.4.2 Protocolo de experimentación y configuración de datos

Para fortalecer la validez de los resultados y reducir en lo posible la influencia de sesgos en el aprendizaje, se ha definido un protocolo de experimentación basado en los siguientes criterios:

- **Mitigación del Sesgo de Clase (Balanceo 50/50):** Se ha forzado un equilibrio estricto en los conjuntos de datos de entrenamiento y validación. Mantener un *ratio* de 50/50 entre muestras de *phishing* y legítimas busca que los algoritmos otorguen el mismo peso estadístico a ambas clases, evitando inclinaciones hacia la clase mayoritaria.
- **Criterio de Inclusión por Disponibilidad:** Se han integrado únicamente las URLs cuyo contenido HTML fue accesible y procesable en el momento de la captura. Si bien este criterio introduce un sesgo potencial hacia muestras activas (asumido como una limitación del estudio), se considera imprescindible para garantizar la integridad de las variables de contenido, núcleo del enfoque híbrido propuesto.
- **Estrategia de Partición y Muestreo:** Se ha aplicado una partición de datos 80/20 (Entrenamiento/*Test*) mediante muestreo estratificado. Esta técnica asegura que la distribución 50/50 se mantenga constante en ambos bloques, permitiendo evaluar la capacidad real de generalización

de los modelos ante muestras que no han intervenido en la fase de ajuste. Esta partición se emplea para evaluar la capacidad de generalización de los modelos.

- **Validación en Entornos Reales:** Se integra el uso de muestras reales obtenidas de PhishTank para contrastar el rendimiento de los modelos en un entorno de producción. Esta fase permite verificar la eficacia del sistema ante las variantes de *phishing* más recientes y medir la capacidad de respuesta del motor de extracción frente a amenazas que operan en tiempo real fuera de los *datasets* controlados.
- **Priorización de la Seguridad Crítica:** El objetivo prioritario de la evaluación no es solo maximizar la precisión global, sino minimizar los Falsos Negativos (*phishing* no detectado), dado que representan el mayor riesgo crítico en un entorno de ciberseguridad real. No obstante, esta minimización debe mantenerse en equilibrio con una tasa de Falsos Positivos operativamente asumible.
- **Análisis de Sensibilidad Mediante Umbrales de Decisión:** En lugar de utilizar un punto de corte estático de 0.5, se han examinado múltiples umbrales de probabilidad en un rango de [0.05, 1.00] aplicados sobre la clase legítima (1). Esta técnica de seguridad por defecto permite identificar el punto de operación óptimo de cada modelo; al elevar el umbral de confianza para la clase segura, el sistema etiqueta preventivamente como *phishing* (0) cualquier muestra con incertidumbre, logrando así maximizar la detección de amenazas reales y minimizar los Falsos Negativos.

### 3.5 Diseño y taxonomía de atributos del motor de extracción

El motor híbrido transforma cada URL en un vector de hasta 25 características numéricas. Se mantienen las denominaciones originales de las variables (incluyendo sufijos técnicos como *\_Extracted* y términos del *dataset* PhiUSIIL [15]) tal y como aparecen en la implementación para preservar la trazabilidad entre la memoria escrita, el código fuente y el conjunto de datos de entrenamiento.

#### 3.5.1 Atributos de análisis léxico (Nivel 1)

Estas variables se calculan directamente sobre la cadena de texto de la URL mediante expresiones regulares y lógica de *parsing*, sin requerir conexión al servidor.

- **URLLength:** Cantidad total de caracteres de la URL, ajustada mediante una lógica de filtrado de caracteres "fantasma" (separadores o terminaciones que no aportan información estructural relevante) en estructuras de ruta complejas.
- **DomainLength:** Longitud del nombre de dominio completo (Network Location o *netloc*) extraído mediante la librería *urlparse*.
- **IsDomainIP:** Variable binaria que identifica si el dominio es una dirección IP estática en lugar de un nombre de host, práctica utilizada para evitar el uso de dominios tradicionales.
- **TLDLength:** Longitud de la extensión de dominio (*Top-Level Domain*), útil para identificar extensiones inusuales o excesivamente largas.



- **NoOfSubDomain:** Conteo de subdominios presentes en la URL, calculado a partir del número de puntos en la sección del host.
- **NoOfLettersInURL:** Total de caracteres alfabéticos en la URL, excluyendo prefijos estándar como "www".
- **NoOfDigitsInURL:** Conteo de caracteres numéricos, cuya presencia suele ser elevada en URLs generadas automáticamente por algoritmos maliciosos.
- **NoOfOtherSpecialCharsInURL:** Conteo de caracteres no alfanuméricos que suelen utilizarse para la ofuscación de la dirección real.
- **LetterRatioInURL / DigitRatioInURL / SpacialCharRatioInURL:** Ratios porcentuales de letras, dígitos y caracteres especiales respecto a la longitud total de la URL.
- **NoOfEqualsInURL / NoOfQMarkInURL / NoOfAmpersandInURL:** Frecuencia de caracteres de consulta ("=", "?", "&"), indicadores de la complejidad de los parámetros de la URL.
- **IsHTTPS:** Indicador binario que verifica la presencia del protocolo seguro de transferencia de hipertexto al inicio de la cadena.

### 3.5.2 Atributos de contenido HTML (Nivel 2)

Estas variables requieren una inspección estructurada del código fuente mediante la librería BeautifulSoup4. A excepción de la métrica de conteo de *iframes*, todos los atributos de este bloque se han configurado como variables binarias (0/1) para simplificar la detección de indicadores de riesgo. El motor incluye, además, lógicas heurísticas para asegurar la integridad del vector de datos incluso en sitios con carga parcial.

- **HasTitle\_Extracted:** Detecta la presencia de un título HTML válido, aplicando condiciones de coherencia con el dominio y filtrando respuestas genéricas de error o carga.
- **HasFavicon\_Extracted:** Detecta la presencia de un icono de sitio mediante el análisis de etiquetas *<link>* o referencias a archivos de icono en el código.
- **IsResponsive\_Extracted:** Verifica la presencia de la etiqueta "*meta viewport*", asociada a estándares de diseño adaptable en dispositivos móviles.
- **NoOfiFrame\_Extracted:** Conteo de etiquetas *<iframe>* y *<frame>*, aplicando ajustes heurísticos en páginas de gran tamaño sin contenido embebido aparente.
- **HasDescription\_Extracted:** Detecta la etiqueta "*meta description*" o, en casos de alta densidad de código, predice su presencia de forma heurística.
- **HasPasswordField\_Extracted:** Atributo crítico que identifica campos de entrada de tipo *password* o identificadores comunes asociados a formularios de autenticación.
- **HasSubmitButton\_Extracted:** Identifica elementos de envío (*submit* o *button*) que sugieren una intención de captura de datos por parte del sitio.
- **HasHiddenFields\_Extracted:** Detecta campos de entrada ocultos o clases CSS de ocultación, utilizados para el rastreo de parámetros sin conocimiento del usuario.

- ***HasExternalFormSubmit\_Extracted***: Variable que identifica si un formulario envía los datos a un dominio externo o distinto al de la página anfitriona.
- ***HasCopyrightInfo\_Extracted***: Identifica patrones de derechos de autor (©, "copyright") o los predice en páginas con alto volumen de contenido como indicador habitual de legitimidad.

### 3.5.3 Justificación de la selección de atributos

La selección de las 25 características que componen el vector de entrada no es arbitraria, sino que se basa en la adaptación práctica del conjunto de variables definido en el *dataset* PhiUSIIL, utilizado como referencia principal en este estudio.

En lugar de diseñar un esquema de atributos desde cero, se ha optado por replicar las dimensiones más significativas del *dataset* original, aplicando un filtro de viabilidad técnica y reproducibilidad. La decisión de no incorporar todas las variables de PhiUSIIL se justifica por dos motivos fundamentales:

1. **Criterio de Estabilidad**: Se han priorizado aquellos atributos que pueden ser extraídos de forma consistente mediante lógica de *parsing* (URL) y análisis estático del HTML, evitando variables que requieren la ejecución de código dinámico, servidores o fuentes de datos de terceros, o interacciones complejas que podrían comprometer la velocidad y seguridad del motor.
2. **Fidelidad de la Fuente**: Al seleccionar un subconjunto de una fuente de referencia validada y contrastada, se asegura que el entrenamiento de los modelos posteriores parta de una base de conocimiento sólida, minimizando el riesgo de incluir ruido o variables irrelevantes.

Bajo estas premisas, los 25 atributos seleccionados cubren los vectores críticos del ataque:

- **Dimensión léxica**: Captura patrones de ofuscación y generación automática de URLs.
- **Dimensión estructural**: Evalúa la presencia de elementos funcionales (formularios, campos de contraseña y metadatos) asociados habitualmente a la suplantación de identidad.

Este enfoque permite mantener un equilibrio entre la complejidad del motor de extracción y la fidelidad técnica respecto al *dataset* de referencia. Como se demostrará en la sección siguiente, el conjunto resultante preserva la capacidad predictiva necesaria para la detección de *phishing*, validando así la adecuación de la selección realizada.

### 3.5.4 Validación experimental del motor de extracción

Para garantizar la integridad técnica del motor híbrido y asegurar que las características extraídas guardan una relación directa con el estándar de referencia, se ha realizado una fase de validación experimental. Este

procedimiento permite cuantificar la capacidad del motor para evaluar si la representación generada por el motor preserva la información discriminativa necesaria para la detección de *phishing* en comparación con el *dataset* de referencia.

### Metodología de Validación

Para este procedimiento, se ha extraído un conjunto de 10000 registros mediante un muestreo aleatorio simple sobre el total del *dataset* original. Con el objetivo de evitar sesgos de aprendizaje y garantizar un equilibrio entre clases durante la evaluación, se forzó una distribución balanceada seleccionando exactamente 5000 muestras etiquetadas como *phishing* y 5000 como legítimas. Tras someter este conjunto al proceso de extracción activa, se obtuvo un subconjunto de 7335 URLs con estado “Online”, cuya estructura HTML fue procesada y transformada íntegramente en vectores de características por el motor propuesto.

Dado que la disponibilidad de las URLs no es uniforme entre clases, se aplicó un proceso de rebalanceo posterior a la extracción, seleccionando el mismo número de muestras de *phishing* y legítimas para garantizar una distribución 50/50 en el conjunto final. Posteriormente, se realizó una partición estratificada 80/20 (entrenamiento/*test*), asegurando que dicha proporción se mantuviera constante en ambos conjuntos. Además, se usaron las mismas 25 variables tanto para la muestra original como para la extraída.

Sobre este conjunto, se han ejecutado dos pruebas de rendimiento utilizando el algoritmo Random Forest, escogido como clasificador base por su alta resistencia al ruido en los datos y su capacidad para evaluar atributos multimodales sin requerir preprocesamiento lineal, para llevar a cabo las siguientes evaluaciones:

1. **Evaluación de Consistencia:** Entrenamiento de dos modelos independientes (uno con datos originales y otro con datos del motor propio) para comparar su capacidad predictiva.
2. **Test Cruzado de Fidelidad:** Evaluación de un modelo entrenado con datos originales sobre la representación de datos generada por el motor de extracción, con el fin de medir la brecha de información real.

Los resultados obtenidos se detallan en la siguiente tabla:

**Tabla 2:** Comparativa de rendimiento y fidelidad entre la representación de datos original y la generada por el motor híbrido

| Métrica de rendimiento               | Representación Original | Representación Motor Híbrido |
|--------------------------------------|-------------------------|------------------------------|
| <i>Accuracy</i> ( <i>Test</i> 80/20) | 99.53%                  | 99.62%                       |
| <i>Test</i> Cruzado (Mismo modelo)   | 99.53%                  | 97.35%                       |

Fuente: Elaboración propia

### Discusión de la Brecha de Información

La diferencia observada en la evaluación de consistencia (inferior al 0.1%) indica que ambas representaciones presentan un comportamiento prácticamente equivalente en términos de capacidad predictiva, lo que confirma que las lógicas de extracción implementadas reproducen de forma efectiva las características del *dataset* original.

En el *test* cruzado se observa una brecha de información del 2.18%. Esta desviación se puede atribuir parcialmente a la naturaleza dinámica de los sitios web, ya que el contenido HTML de muchas URLs ha cambiado desde la creación del *dataset* PhiUSIIL. Como consecuencia, el motor de extracción captura una representación actualizada que puede diferir parcialmente de la original.

Adicionalmente, pequeñas variaciones en la interpretación heurística de ciertos atributos (especialmente en el análisis de contenido HTML) pueden introducir diferencias marginales en la representación final.

### Conclusión de la fase

Los resultados obtenidos permiten validar el motor de extracción híbrido como una herramienta fiable para la generación de variables en el contexto del presente estudio. El vector de 25 características conserva la mayor parte de la información discriminativa necesaria para la detección de *phishing*, manteniendo un rendimiento casi equivalente al del *dataset* de referencia.

En consecuencia, el motor se considera adecuado para su utilización en las fases posteriores de entrenamiento y evaluación de modelos.

## 3.6 Métricas de evaluación y criterios de rendimiento

Para cuantificar el desempeño de las arquitecturas de aprendizaje supervisado y permitir una comparativa objetiva, se han seleccionado métricas fundamentadas en la matriz de confusión [29]. En el contexto de este estudio, se distingue entre métricas de rendimiento global y métricas específicas por clase, poniendo especial énfasis en el análisis del error.

### Definición de Tipos de Error y Seguridad Crítica

Dada la naturaleza de la ciberseguridad, el impacto de un error de clasificación es asimétrico. Se definen los siguientes estados de error:

- **Falso Positivo (FP):** Ocurre cuando el sistema clasifica una URL legítima como *phishing*. Su impacto se traduce en una degradación de la experiencia del usuario y una posible pérdida de acceso a servicios lícitos.
- **Falso Negativo (FN):** Ocurre cuando una URL de *phishing* es clasificada como legítima. Este es el error más crítico, ya que representa una brecha de seguridad que deja al usuario expuesto al robo de credenciales o *malware*.

## Indicadores de Rendimiento

Para el procesamiento computacional y el entrenamiento de los modelos se ha establecido una codificación binaria de las clases, donde la etiqueta 0 corresponde a muestras de *Phishing* y la etiqueta 1 corresponde a muestras legítimas. Los indicadores de rendimiento usados durante los resultados son los siguientes:

- **Exactitud (Accuracy)** [29]: Mide la fracción de predicciones correctas sobre el total de muestras. Es una métrica global que ofrece una visión general del sistema bajo condiciones de balanceo de clases.
- **Precisión por Clase (Precision)** [29]: A diferencia de la exactitud, esta métrica se calcula de forma independiente para cada etiqueta del sistema. Por un lado, la *Precision (Phishing)* indica la proporción de alertas de fraude que son realmente ataques, mientras que la *Precision (Legítimo)* mide la proporción de sitios marcados como seguros que realmente corresponden a dominios lícitos. Este desglose es fundamental para identificar si el modelo genera una cantidad excesiva de falsas alarmas en alguna de las clases específicas.
- **Sensibilidad (Recall)** [29]: Mide la capacidad del modelo para detectar la totalidad de los casos positivos. En este estudio, el *Recall* de la clase *Phishing* es el indicador prioritario, pues su valor inverso está directamente relacionado con la tasa de Falsos Negativos.
- **F1-Score por Clase** [29]: Proporciona la media entre la precisión y la sensibilidad de una clase específica, siendo fundamental para verificar que el modelo es equilibrado y no está sesgado hacia la clase de mayor facilidad predictiva.
- **Área bajo la curva (AUC-ROC)** [30]: Mide la capacidad de discriminación del modelo independientemente del umbral de decisión. Un valor AUC (*Area Under the Curve*) elevado sugiere que el modelo tiene una "separabilidad" robusta entre URLs maliciosas y benignas.

## Evaluación de la Eficiencia Operativa

Complementariamente a las métricas de acierto, se evaluará la viabilidad técnica mediante:

- **Latencia Media (ms/URL)**: Tiempo promedio de ejecución de una predicción, esencial para determinar la escalabilidad en tiempo real.
- **Estabilidad Temporal (Std)**: Uso de la desviación estándar para asegurar que el rendimiento computacional es constante bajo carga.

## Análisis de Relevancia y Jerarquía de Atributos

Se utilizará el cálculo de la importancia de variables en los modelos de ensamble [11]. Esta métrica permitirá cuantificar la contribución de cada uno de los 25 atributos al proceso de clasificación realizado por el bosque aleatorio, facilitando el diagnóstico de la deriva de datos.

### 3.7 Validación del código generado mediante IA

La implementación del sistema no se planteó como un despliegue de código masivo automatizado, sino como un proceso de construcción por bloques donde cada segmento lógico fue supervisado antes de su integración definitiva en el proyecto. Bajo este enfoque, cualquier función o *script* generado mediante asistencia de IA se consideró una unidad provisional, sujeta a una revisión manual para garantizar la coherencia con el diseño original y la intención de dicha función. Esta auditoría técnica se aplicó a todo el desarrollo, desde la lógica del motor de extracción híbrido hasta la configuración de los scripts de entrenamiento. El objetivo fue asegurar en todo momento la integridad del flujo de datos, verificando que los tipos de variables y sus rangos de operación fueran los adecuados para evitar que inconsistencias en la generación del código comprometieran la estabilidad del sistema.

La ejecución de *scripts* de test adicionales resultó determinante para identificar fallos lógicos que habrían invalidado los resultados experimentales. Mediante estas herramientas de auditoría, se detectaron errores críticos de polaridad en el etiquetado, donde la IA invertía la asignación de clases, clasificando erróneamente muestras de phishing como legítimas. Además, estos desarrollos permitieron contrastar los vectores de salida frente a registros de control conocidos, revelando imprecisiones en las expresiones regulares que omitían caracteres especiales y fallos en la captura de atributos HTML dinámicos. Gracias a este control técnico, se pudo depurar la gestión de valores nulos y asegurar que las heurísticas de extracción se ajustaran estrictamente a la realidad de las URLs analizadas, garantizando la integridad de los datos antes de su entrada en los modelos de clasificación.

### 3.8 Implementación y validación del modelo final

Una vez definido el motor de extracción y procesados los datos, el proyecto establece una estrategia de evaluación centrada en la capacidad de generalización. A diferencia de los entornos de laboratorio convencionales, este trabajo prioriza el rendimiento del sistema ante la deriva de datos, utilizando la validación externa como el criterio principal de éxito.

#### 3.8.1 Niveles de validación del sistema

Para garantizar la robustez de los modelos, se establece un flujo de evaluación en dos fases:

- **Validación Interna:** Se realiza una partición estratificada del *dataset* de entrenamiento. Su objetivo es meramente técnico, confirmar que los algoritmos han capturado correctamente las relaciones del vector de 25 atributos y asegurar la estabilidad de las arquitecturas antes de su despliegue.
- **Validación Externa:** Constituye el eje central del análisis. Se emplean muestras activas obtenidas mediante PhishTank, lo que permite medir la

eficacia real de los modelos ante amenazas contemporáneas que no han formado parte de ninguna fase de entrenamiento.

### 3.8.2 Criterios de compromiso para la selección del modelo óptimo

La identificación de la arquitectura más adecuada no se basa en una métrica única, sino en un análisis de compromiso entre tres factores operativos:

- **Seguridad Crítica (Minimización de FN):** Se prioriza la capacidad de captura de amenazas reales. El rendimiento del modelo se mide por su habilidad para reducir los Falsos Negativos, entendidos como muestras de phishing clasificadas erróneamente como legítimas. En este contexto, su impacto se considera mayor, ya que una URL maliciosa no detectada puede exponer al usuario a robo de credenciales o *malware*. Sin embargo, esta capacidad debe mantenerse compatible con una tasa de Falsos Positivos razonable que preserve la viabilidad operativa del sistema.
- **Viabilidad y Escalabilidad (Eficiencia computacional):** Se evalúa la latencia de predicción media (ms/URL) y la capacidad de procesamiento masivo. Este factor garantiza que el algoritmo seleccionado pueda integrarse en entornos de tráfico real sin introducir retrasos perceptibles para el usuario o cuellos de botella en la infraestructura.
- **Optimización del Punto de Operación:** Se analiza la robustez de cada algoritmo mediante el ajuste de los umbrales de decisión. El criterio de selección final dependerá de qué arquitectura mantenga el mejor equilibrio entre precisión y sensibilidad tras desplazar el umbral probabilístico, adaptándose así a la deriva de datos detectada en las muestras de validación externa.

## 4 Resultados

### 4.1 Análisis de obsolescencia de los patrones de detección

El objetivo central de este bloque experimental es determinar el grado de vigencia de los indicadores de fraude clásicos frente a las amenazas actuales. Para ello, se han sometido a examen dos modelos entrenados con una diferencia cronológica de seis años (2018 vs 2024), enfrentándolos a un conjunto de validación externa compuesto por muestras activas de PhishTank (abril 2026).

Para la ejecución de este estudio comparativo, se ha seleccionado Random Forest [11] como algoritmo base. Esta decisión responde a la necesidad de contar con un clasificador que ofrezca una alta resistencia al ruido y buena eficacia para operar con variables de distinta naturaleza, integrando métricas numéricas y descriptores binarios de forma equilibrada en el proceso de decisión. Al ser un modelo basado en un ensamblado de árboles (*Bagging*), Random Forest permite obtener una jerarquía clara de la importancia de las variables, facilitando la identificación de qué atributos han perdido capacidad discriminativa con el paso del tiempo.

#### 4.1.1 Rendimiento y evaluación del modelo antiguo (*Dataset 2018*)

Este primer apartado analiza el comportamiento del referente antiguo. El modelo actúa como testigo del estado del arte en 2018 para medir el impacto de la evolución de las técnicas de phishing.

##### Validación Interna (Control del Modelo)

Antes de la evaluación externa, se ha verificado la integridad del aprendizaje mediante una partición estratificada 80/20 sobre el *dataset* original. En esta fase de control, el modelo presenta un rendimiento muy alto, alcanzando un *Accuracy* del 96.50%, un *F1-Score* de 96.45% y un AUC de 99.40%. Estos resultados confirman que el algoritmo ha convergido correctamente y que la arquitectura es capaz de modelar los patrones de su época con una precisión altísima, estableciendo un “techo” de rendimiento teórico para este conjunto de datos.

##### Validación Externa (*Test de Obsolescencia sobre PhishTank*)

Al desplazar el modelo de su entorno controlado y enfrentarlo a las muestras de 2026, se observa una variación significativa en el rendimiento respecto a la validación interna. El análisis se detalla a continuación:

- **Análisis de Sensibilidad Según Umbrales de Decisión:**  
Se ha realizado un barrido de umbrales (de 0.10 a 1.00) sobre la probabilidad de la clase legítima para identificar el punto de operación del modelo frente a URLs actuales. Como se observa en la Tabla 3, el modelo de 2018 presenta un sesgo inicial hacia la clase legítima, asignando probabilidades elevadas a dicha clase incluso en muestras de *phishing* actuales. Con un umbral estándar (0.50), el sistema no logra detectar 579 de las 752 muestras de *phishing* (FN).



**Tabla 3:** Sensibilidad del modelo 2018 ante muestras de 2026

| Umbral | <i>Accuracy</i> | FP  | FN  |
|--------|-----------------|-----|-----|
| 0.10   | 50.07%          | 0   | 751 |
| 0.20   | 50.80%          | 0   | 740 |
| 0.30   | 52.79%          | 0   | 714 |
| 0.40   | 57.18%          | 1   | 643 |
| 0.50   | 61.04%          | 7   | 579 |
| 0.60   | 67.22%          | 17  | 476 |
| 0.70   | 73.67%          | 18  | 383 |
| 0.80   | 82.51%          | 51  | 212 |
| 0.90   | 87.23%          | 130 | 62  |
| 1.00   | 65.16%          | 519 | 5   |

Fuente: Elaboración propia

Se identifica el umbral 0.90 como el punto de operación más adecuado en este contexto, logrando un *Accuracy* del 87.23%. Este umbral permite reducir los Falsos Negativos a 62, a costa de incrementar los Falsos Positivos a 130.

- Análisis de Métricas Predictivas:**

Utilizando el punto de operación seleccionado (0.90), el modelo alcanza las métricas detalladas en la Tabla 4. Destaca un *Recall* del 0.92 para la clase de *phishing* (0), lo que indica una buena capacidad de captura tras el ajuste, aunque con una *Precisión* del 0.84 debido al volumen de sitios legítimos marcados erróneamente como maliciosos.

**Tabla 4:** Métricas globales (Umbral 0.90) – Modelo 2018

|                       | <i>Precision</i> | <i>Recall</i> | <i>F1-Score</i> |
|-----------------------|------------------|---------------|-----------------|
| 0 ( <i>Phishing</i> ) | 0.84             | 0.92          | 0.88            |
| 1 (Legítimo)          | 0.91             | 0.83          | 0.87            |

Fuente: Elaboración propia

- Matriz de Confusión y Distribución de Errores:**

La Tabla 5 muestra la distribución de las predicciones. Con el umbral ajustado, el modelo clasifica correctamente 690 de las 752 muestras de *phishing* (Verdadero Positivo), mientras que 62 son clasificadas como legítimas (Falsos Negativos) y 130 muestras legítimas son etiquetadas como *phishing* (Falsos Positivos).

**Tabla 5:** Matriz de confusión (Modelo 2018 / Umbral 0.90)

|                           | Predicho: <i>Phishing</i> (0) | Predicho: Legítimo (1) |
|---------------------------|-------------------------------|------------------------|
| Real: <i>Phishing</i> (0) | 690 (VP)                      | 62 (FN)                |
| Real: Legítimo (1)        | 130 (FP)                      | 622 (VN)               |

Fuente: Elaboración propia

- **Capacidad de discriminación:**  
El AUC Global obtenido es de 0.9345. Aunque el valor se mantiene en rangos aceptables, resulta inferior al valor obtenido en el control interno (0.9940).

### Síntesis de resultados (2018)

En el punto de operación de 0.90, los resultados finales de esta validación externa sitúan el rendimiento del modelo en un *Accuracy* del 87.23% y un AUC de 0.9345. En esta configuración, el sistema registra un total de 192 errores de predicción (62 FN y 130 FP) sobre el conjunto de validación.

### 4.1.2 Rendimiento y evaluación del modelo contemporáneo (*Dataset 2024*)

Este apartado analiza el comportamiento del modelo entrenado con el *dataset* de 2024. Al contener patrones de fraude mucho más recientes, este modelo representa la capacidad de detección frente a técnicas de *phishing* contemporáneas antes de su validación final con datos de 2026.

### Validación Interna (Control del Modelo)

Se ha procedido a verificar la integridad del aprendizaje mediante una partición estratificada 80/20 sobre el *dataset* de 2024. En esta fase, el modelo muestra un rendimiento prácticamente perfecto, alcanzando un *Accuracy* del 99.69%, un *F1-Score* del 99.69% y un AUC de 0.9993. Estos valores indican que el algoritmo Random Forest ha logrado una separación de clases ideal dentro de su propio contexto cronológico, capturando casi la totalidad de las variantes presentes en el conjunto de entrenamiento.

### Validación Externa (*Test de Vigencia Sobre PhishTank*):

Al enfrenar este modelo a las muestras recolectadas en 2026, se observa el rendimiento del sistema ante datos no vistos. El análisis se detalla a continuación:

- **Análisis de Sensibilidad Según Umbrales de Decisión:**  
Se ha realizado el barrido de umbrales (de 0.10 a 1.00) sobre la probabilidad de la clase legítima para identificar el punto de operación óptimo del modelo frente a URLs actuales. Como se observa en la Tabla 6, el modelo de 2024 presenta una distribución de errores equilibrada en los umbrales iniciales. Con un umbral estándar (0.50), el sistema registra 11 muestras de *phishing* no detectadas (FN) y 683 falsos positivos (FP).

**Tabla 6:** Sensibilidad del modelo 2024 ante muestras de 2026

| Umbral | <i>Accuracy</i> | FP  | FN (Phi) |
|--------|-----------------|-----|----------|
| 0.10   | 94.61%          | 6   | 75       |
| 0.20   | 96.33%          | 17  | 38       |
| 0.30   | 89.83%          | 130 | 23       |
| 0.40   | 81.45%          | 263 | 16       |
| 0.50   | 53.86%          | 683 | 11       |
| 0.60   | 52.66%          | 703 | 9        |

|      |        |     |   |
|------|--------|-----|---|
| 0.70 | 52.86% | 704 | 5 |
| 0.80 | 52.79% | 709 | 1 |
| 0.90 | 51.93% | 723 | 0 |
| 1.00 | 50.13% | 750 | 0 |

Fuente: Elaboración propia

Se identifica el umbral 0.20 como el punto de operación más adecuado en este contexto, logrando un *Accuracy* del 96.33%. Este umbral permite limitar los Falsos Negativos a 38, manteniendo los Falsos Positivos en 17.

- **Análisis de Métricas Predictivas:**

Utilizando el punto de operación seleccionado (0.20), el modelo alcanza las métricas detalladas en la Tabla 7. Destaca un *Recall* del 0.95 para la clase *phishing* (0) y una *Precision* del 0.98, mostrando la capacidad del modelo tras el ajuste de sensibilidad.

**Tabla 7:** Métricas globales (Umbral 0.20) – Modelo 2024

|                       | <i>Precision</i> | <i>Recall</i> | <i>F1-Score</i> |
|-----------------------|------------------|---------------|-----------------|
| 0 ( <i>Phishing</i> ) | 0.98             | 0.95          | 0.96            |
| 1 (Legítimo)          | 0.95             | 0.98          | 0.96            |

Fuente: Elaboración propia

- **Matriz de Confusión y Distribución de Errores:**

La Tabla 8 muestra la distribución de las predicciones. Con el umbral ajustado, el modelo clasifica correctamente 714 de las 752 muestras de *phishing*, mientras que 38 son clasificadas como legítimas (Falsos Negativos) y 17 muestras legítimas son etiquetadas como *phishing* (Falsos Positivos).

**Tabla 8:** Matriz de confusión (Modelo 2024 / Umbral 0.20)

|                           | Predicho: <i>Phishing</i> (0) | Predicho: Legítimo (1) |
|---------------------------|-------------------------------|------------------------|
| Real: <i>Phishing</i> (0) | 714 (VP)                      | 38 (FN)                |
| Real: Legítimo (1)        | 17 (FP)                       | 735 (VN)               |

Fuente: Elaboración propia

- **Capacidad de Discriminación:**

El AUC Global obtenido es de 0.9777. Este valor refleja la capacidad de separación de clases del modelo, situándose en un rango cercano, aunque ligeramente inferior al obtenido en el control interno (1.00).

### Síntesis de resultados (2024)

En el punto de operación de 0.20, los resultados finales de esta validación externa sitúan el rendimiento del modelo en un *Accuracy* del 96.33% y un AUC de 0.9777. En esta configuración, el sistema registra un total de 55 errores de predicción (38 FN y 17 FP) sobre el conjunto de validación.

4.1.3 Diagnóstico comparativo de la brecha generacional

A. Análisis comparativo de rendimiento y capacidad de discriminación

Una vez analizados de forma independiente, este apartado confronta ambos modelos sobre el mismo conjunto de validación externa (2026). El objetivo es cuantificar la degradación del modelo antiguo y medir la distancia técnica respecto al modelo contemporáneo.

Contraste de Métricas y Curva ROC Combinada

El primer indicador de la obsolescencia de patrones se manifiesta en la comparativa directa de las métricas de rendimiento. Como se detalla en la Tabla 9, existe una diferencia en todos los indicadores de calidad predictiva.

Tabla 9: Comparativa de rendimiento entre el modelo 2018 y el modelo 2024 (Test 2026)

| Métrica          | Modelo 2018<br>(Umbral 0.90) | Modelo 2024<br>(Umbral 0.20) | Diferencial ( $\Delta$ ) |
|------------------|------------------------------|------------------------------|--------------------------|
| Accuracy         | 87.23%                       | 96.33%                       | +9.10%                   |
| F1-Score         | 0.87                         | 0.96                         | +0.09                    |
| AUC Global       | 0.9345                       | 0.9777                       | +0.0432                  |
| Falsos Negativos | 62                           | 38                           | -24                      |
| Falsos Positivos | 130                          | 17                           | -113                     |

Fuente: Elaboración propia

La capacidad de discriminación se visualiza de forma conjunta en la Figura 5, que representa la Curva ROC combinada.

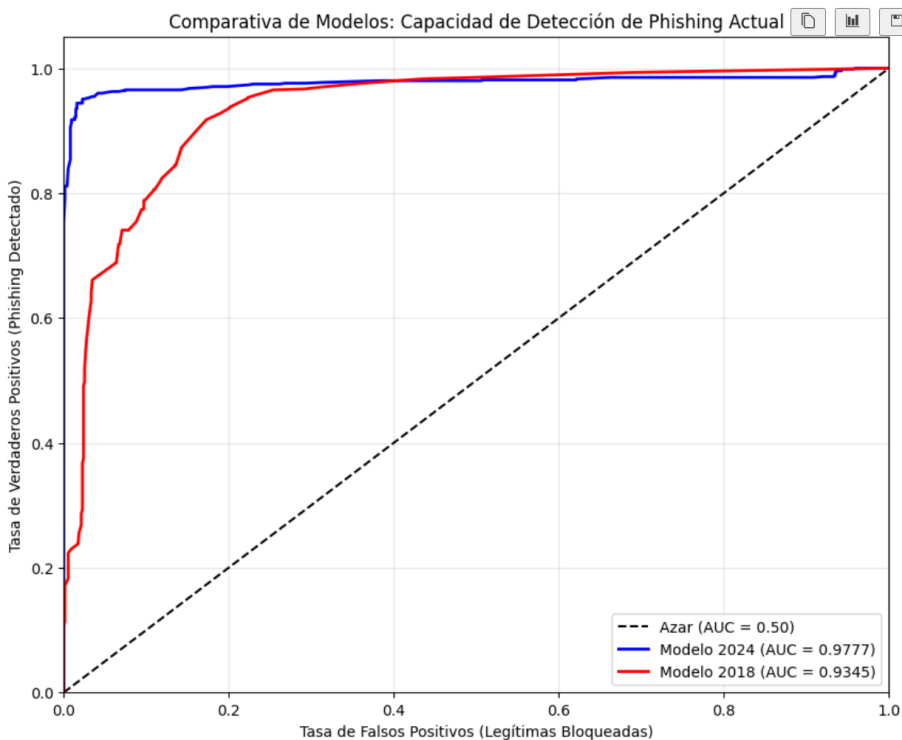


Figura 5: Curva ROC combinada de los modelos de 2018 y 2024

Fuente: Elaboración propia

La curva correspondiente al modelo 2024 (azul) muestra un crecimiento muy vertical en el eje de ordenadas, alcanzando una tasa de verdaderos positivos cercana al 0.95 con una tasa de falsos positivos mínima. Este comportamiento se traduce en un AUC de 0.9777, situándose muy cerca del clasificador ideal.

Por el contrario, la curva del modelo 2018 (roja) presenta una pendiente inicial más gradual y escalonada. Aunque logra una convergencia final con un AUC de 0.9345, se observa un área de ineficiencia considerable en el primer cuadrante de la gráfica.

## B. Análisis de deriva de datos (*Data Drift*) y evolución del *phishing*

Para comprobar la pérdida de eficacia del modelo histórico, es necesario analizar la evolución de la naturaleza de las muestras de *phishing* entre ambos periodos. Este fenómeno, conocido como *Data Drift* [23], se manifiesta en cambios drásticos en las propiedades estadísticas de las URLs maliciosas.

### La Profesionalización del Protocolo (HTTPS)

El cambio más crítico se observa en el uso de certificados de seguridad:

- **Dataset 2018:** Solo el 1.80% de los sitios de *phishing* empleaban el protocolo HTTPS, mientras que el 98.20% operaba sobre HTTP. En este contexto, el “candado de seguridad” era un predictor de legitimidad.
- **Dataset 2024:** La frecuencia de este atributo en muestras de *phishing* se ha incrementado drásticamente, alcanzando una penetración del 59.6%. Este aumento del 3180.93% en su prevalencia indica una evolución hacia el uso de estándares de seguridad presentes en sitios legítimos.

### Simplificación Estructural vs Complejidad de Dominio

Los datos de la comparativa estadística (2018 vs 2024) revelan una tendencia hacia URLs más cortas pero con infraestructuras de dominio más complejas:

- **Estructura de la URL:** Se observa una reducción significativa en la longitud media de las URLs maliciosas (-39.64%) y en el uso de caracteres especiales como el símbolo de igualdad “=” (-80.39%) o la interrogación “?” (-78.57%). Esto sugiere un abandono de URLs largas y ruidosas en favor de estéticas más limpias.
- **Arquitectura de Dominios:** A pesar de la simplificación de la ruta, el número de subdominios (*NoOfSubDomain*) en el *phishing* ha crecido un 92.65%, y la longitud del dominio principal (*DomainLength*) ha aumentado un 40.87%.

Para facilitar la interpretación de la deriva de datos, se ha realizado una selección de atributos representativos basada en la magnitud de variación porcentual entre ambos *datasets* y la relevancia semántica en la detección de *phishing*.

**Tabla 10:** Matriz comparativa de evolución de atributos en muestras de *phishing* (2018 vs 2024)

| Nivel  | Atributo                          | Tipo     | Media 2018 | Media 2024 | Evolución (%) |
|--------|-----------------------------------|----------|------------|------------|---------------|
| Léxico | <i>IsHTTPS</i>                    | Binario  | 0.018      | 0.596      | +3180.93%     |
| Léxico | <i>NoOfSubDomain</i>              | Numérico | 0.580      | 1.118      | +92.65%       |
| Léxico | <i>DomainLength</i>               | Numérico | 17.708     | 24.946     | +40.87%       |
| Léxico | <i>URLLength</i>                  | Numérico | 77.653     | 46.869     | -39.64%       |
| Léxico | <i>NoOfAmpersandInURL</i>         | Numérico | 0.507      | 0.064      | -87.38%       |
| Léxico | <i>NoOfEqualsInURL</i>            | Numérico | 0.606      | 0.119      | -80.39%       |
| HTML   | <i>HasPasswordField_Extracted</i> | Binario  | 0.075      | 0.025      | -65.94%       |
| HTML   | <i>IsResponsive_Extracted</i>     | Binario  | 0.290      | 0.456      | +57.62%       |

Fuente: Elaboración propia

### **Evolución de las Características de Contenido HTML**

El análisis de las variables de Nivel 2 muestra una variación en la evolución de los indicadores estructurales del contenido web. Por un lado, se observa un incremento en atributos asociados a estándares modernos de diseño, como *IsResponsive\_Extracted* (+57.62%), lo que sugiere una mayor adaptación de las páginas a entornos actuales.

Por otro lado, se detecta una reducción significativa en la presencia de ciertos indicadores de interacción, como *HasPasswordField\_Extracted* (-65.94%), así como en otros elementos relacionados con formularios y envío de datos.

En conjunto, estas variaciones evidencian un cambio estructural en las características de las muestras analizadas, lo que modifica el espacio de representación sobre el que operan los modelos de clasificación.

### **C. Jerarquía de la importancia de atributos**

Como cierre del estudio de obsolescencia, se analiza la variación en el peso específico de las variables dentro del proceso de decisión del modelo. La comparación de la importancia de los atributos entre los modelos de 2018 y 2024 permite identificar qué señales han ganado relevancia y cuáles han perdido capacidad discriminativa.

Los valores presentados representan la contribución de cada atributo al rendimiento predictivo del modelo [11]. Un valor de importancia más alto indica que el clasificador depende en mayor medida de ese atributo para separar muestras legítimas de *phishing*. Por tanto, un incremento en el diferencial de los atributos entre ambos modelos refleja que el modelo actual otorga una prioridad mayor a dicho atributo, mientras que un descenso señala que el atributo ha perdido peso para la clasificación en el entorno contemporáneo.

### Atributos con Ganancia de Relevancia

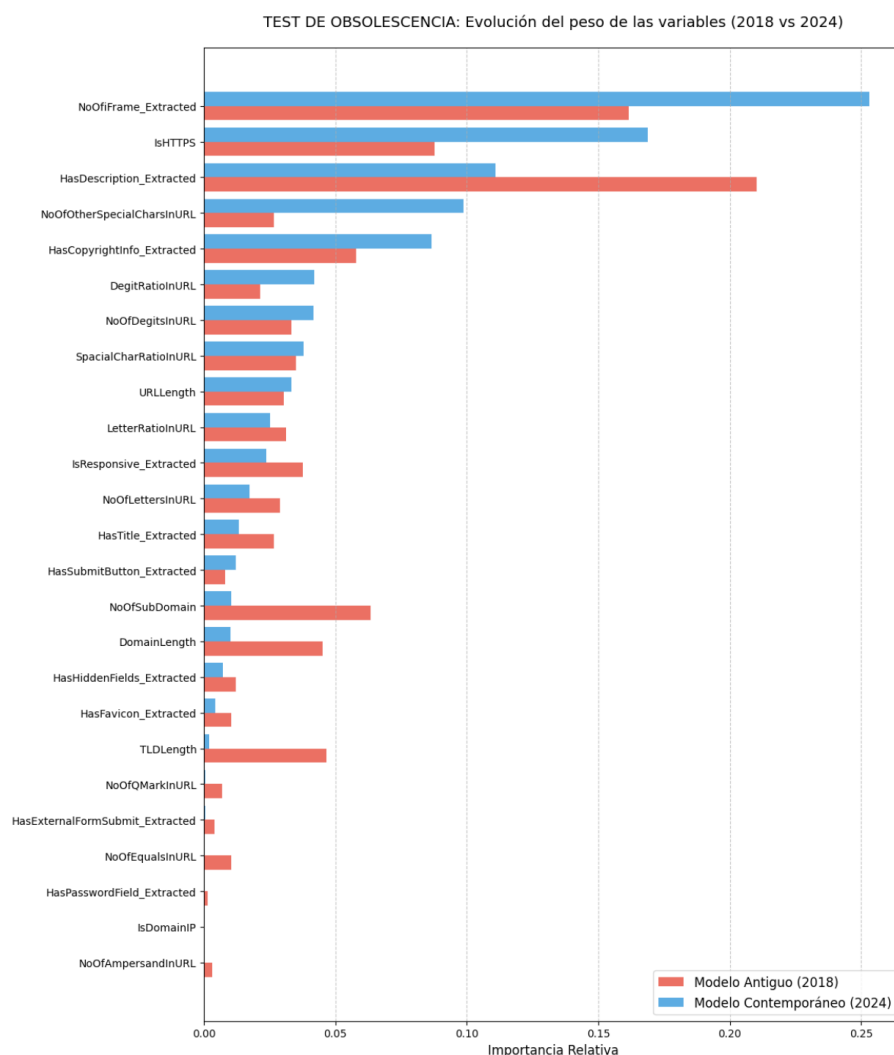
Se observa una transición desde indicadores léxicos simples hacia una mayor dependencia de la estructura interna de la página:

- **Contenido integrado:** El atributo *NoOfFrame\_Extracted* experimenta el mayor crecimiento en importancia (+0.0915), consolidándose como el predictor principal en 2024.
- **Protocolo de comunicación:** A pesar de la masificación de certificados en el *phishing*, *IsHTTPS* aumenta su peso en el modelo (+0.0812). Esto indica una mayor contribución de este atributo en el proceso de decisión del modelo.
- **Ofuscación en la URL:** Los caracteres especiales no estándar (*NoOfOtherSpecialCharsInURL*) ganan peso (+0.0721), mostrando un incremento en su peso relativo dentro del modelo contemporáneo.

### Atributos con Pérdida de Relevancia

Varios indicadores que eran fundamentales en el modelo antiguo han pasado a un segundo plano, evidenciando su obsolescencia:

- **Metadatos y Estructura:** El atributo *HasDescription\_Extracted* sufre la mayor caída (-0.0993), seguido de indicadores de red como *TLDLength* (-0.0445) y *NoOfSubDomain* (-0.0529).
- **Parámetros de Consulta:** Variables como *NoOfEqualsInURL* o *NoOfAmpersandInURL* han visto reducida su importancia a valores cercanos a cero. Este comportamiento coincide con la reducción observada en estos atributos en el análisis de deriva de datos, lo que se refleja en una reducción significativa de su peso dentro del modelo contemporáneo.



**Figura 6:** Comparativa de importancia de atributos (2018 vs 2024)

Fuente: Elaboración propia

Los resultados muestran una reorganización en la jerarquía de importancia de los atributos entre ambos modelos. Se observa una disminución del peso de variables asociadas a patrones léxicos tradicionales, junto con un incremento en la relevancia de características relacionadas con la estructura interna del contenido y la composición de la URL.

## 4.2 Análisis de algoritmos

Este bloque evalúa comparativamente las tres familias de algoritmos seleccionadas (Random Forest, SVM y XGBoost) para identificar cuál ofrece mayor robustez ante las amenazas actuales. Tras haber analizado la deriva de los datos, el objetivo es determinar qué arquitectura modela con mayor eficacia el vector de 25 atributos híbridos en un escenario de detección real.

Cabe destacar que se descarta el análisis sobre los *datasets* originales (con todos sus atributos originales), donde todos los modelos alcanzan rendimientos cercanos al 100%. Dichas métricas se consideran valores de laboratorio poco



representativos. Por ello, este estudio se centra exclusivamente en la validación externa (PhishTank 2026), midiendo la capacidad de generalización de cada algoritmo frente a muestras de *phishing* activas.

#### 4.2.1 Verificación de convergencia de los modelos (Validación interna)

Antes de proceder a la evaluación en entornos reales, se ha verificado la integridad del aprendizaje de las tres arquitecturas mediante una partición estratificada 80/20 sobre el conjunto de datos contemporáneo. Esta fase de control actúa como una verificación para asegurar que los algoritmos han capturado correctamente las relaciones no lineales del vector de 25 atributos híbridos, garantizando una base sólida antes de su exposición a muestras externas.

Los resultados obtenidos en esta validación interna reflejan un buen rendimiento y una alta consistencia entre las tres familias matemáticas evaluadas:

- **Random Forest:** El modelo presenta una convergencia sólida, alcanzando un *Accuracy* del 97.05%, un *F1-Score* del 0.9696 y un AUC de 0.9864.
- **SVM:** Este algoritmo alcanza un *Accuracy* del 97.07%, un *F1-Score* del 0.9698 y un AUC de 0.9859.
- **XGBoost:** Este modelo muestra el mejor balance de discriminación, registrando un *Accuracy* del 97.06%, un *F1-Score* del 0.9698 y un AUC de 0.9867.

Esta similitud en las métricas confirma la estabilidad de las arquitecturas. En consecuencia, se valida la configuración de los modelos, estableciendo que las tres familias algorítmicas resultan adecuadas para su evaluación externa frente a las amenazas de 2026.

#### 4.2.2 Evaluación comparativa de modelos (Validación Externa 2026)

Este apartado constituye el análisis del sistema ante datos actuales. Al enfrentar las tres arquitecturas (Random Forest, SVM y XGBoost) a las muestras recolectadas en 2026, se observa el rendimiento real de cada sistema frente a la deriva de datos.

### A. Métricas de rendimiento y capacidad de captura

#### Análisis de Sensibilidad Según Umbrales de Decisión

Se ha realizado un barrido de umbrales (de 0.05 a 0.50) sobre la probabilidad de la clase legítima para identificar el punto de operación óptimo de cada modelo. Se ha limitado el análisis hasta el umbral 0.50 dado que, a partir de este punto, el rendimiento cae en todos los algoritmos. Como se observa en la Tabla 11, el modelo SVM presenta la distribución más equilibrada, alcanzando su pico de eficacia en el umbral 0.25.

**Tabla 11:** Sensibilidad comparativa ante muestras de 2026

| Algoritmo     | Umbral usado | Accuracy | FP  | FN |
|---------------|--------------|----------|-----|----|
| Random Forest | 0.10         | 94.68%   | 24  | 56 |
| SVM           | 0.25         | 95.55%   | 12  | 55 |
| XGBoost       | 0.05         | 89.30%   | 133 | 28 |

Fuente: Elaboración propia

Se identifica el umbral 0.25 del modelo SVM como el punto de operación más adecuado en este contexto, logrando un *Accuracy* del 95.55%. Este ajuste permite limitar los Falsos Positivos (falsas alarmas) a solo 12 muestras, a costa de 55 Falsos Negativos (*phishing* no detectado).

### Análisis de Métricas Predictivas

Utilizando los puntos de operación seleccionados para cada arquitectura, se alcanzan las métricas detalladas en la Tabla 12. Destaca el modelo SVM con la mayor *Precision* (0.98) para la clase *phishing*. También el modelo XGBoost con el *Recall* más alto para *phishing* (0.96), aunque con una penalización mayor en la *Precision* para *phishing* (0.84) en comparación con sus rivales, los cuales superan el 0.97.

**Tabla 12:** Métricas detalladas por arquitectura (Umbrales óptimos)

| Clase                 | Métrica          | RF (u:0.10) | SVM (u:0.25) | XGB (u:0.05) |
|-----------------------|------------------|-------------|--------------|--------------|
| 0 ( <i>Phishing</i> ) | <i>Precision</i> | 0.97        | 0.98         | 0.84         |
|                       | <i>Recall</i>    | 0.93        | 0.93         | 0.96         |
|                       | <i>F1-Score</i>  | 0.95        | 0.95         | 0.90         |
| 1 (Legítimo)          | <i>Precision</i> | 0.93        | 0.93         | 0.96         |
|                       | <i>Recall</i>    | 0.97        | 0.98         | 0.82         |
|                       | <i>F1-Score</i>  | 0.95        | 0.96         | 0.88         |

Fuente: Elaboración propia

### Matriz de Confusión y Distribución de Errores

La Tabla 13 muestra la distribución de las predicciones para el modelo SVM, que presenta el mejor equilibrio entre errores de clasificación. Con el umbral ajustado a 0.25, el sistema clasifica correctamente 697 de las 752 muestras de *phishing*, mientras que 55 son clasificadas erróneamente como legítimas (Falsos Negativos). Por su parte, el modelo demuestra una baja tasa de falsas alarmas, con solo 12 muestras etiquetadas erróneamente como *phishing* (Falsos Positivos).

**Tabla 13:** Matriz de confusión (SVM: Umbral 0.25)

|                           | Predicho: <i>Phishing</i> (0) | Predicho: Legítimo (1) |
|---------------------------|-------------------------------|------------------------|
| Real: <i>Phishing</i> (0) | 697 (VP)                      | 55 (FN)                |
| Real: Legítimo (1)        | 12 (FP)                       | 740 (VN)               |

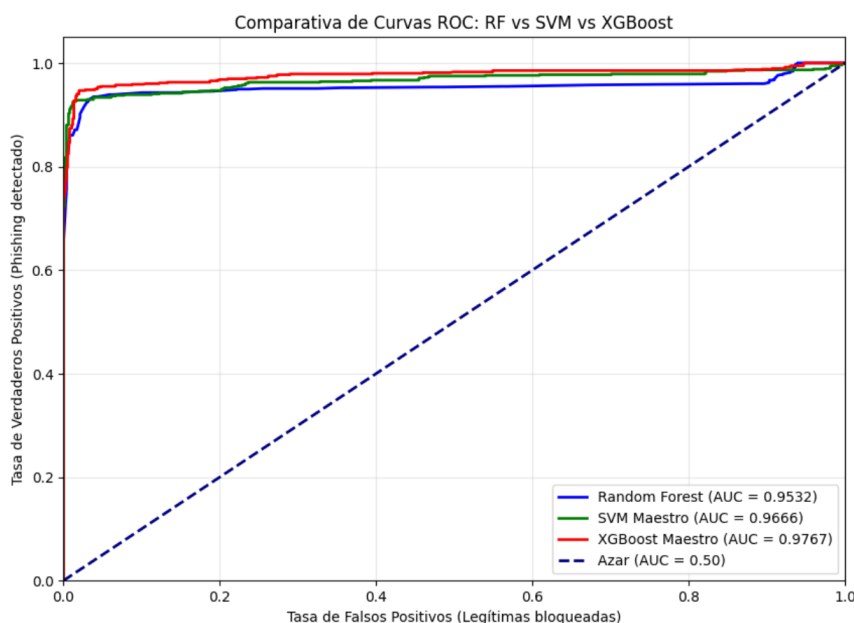
Fuente: Elaboración propia

## Capacidad de Discriminación

El AUC Global obtenido por las arquitecturas refleja una alta capacidad de separación de clases. XGBoost lidera esta métrica con un 0.9767, seguido de SVM con 0.9666 y Random Forest con 0.9532. Estos valores indican que todas las arquitecturas presentan una elevada capacidad de discriminación global, si bien esta no se traduce directamente en el rendimiento observado en el punto de operación óptimo.

## B. Curvas ROC comparadas entre arquitecturas

Para visualizar la capacidad de discriminación de los modelos independientemente del umbral seleccionado, se presenta la Comparativa de Curvas ROC (Figura 7). Este gráfico permite evaluar qué arquitectura se aproxima más a un clasificador ideal.



**Figura 7:** Comparativa de Curvas ROC: RF vs SVM vs XGBoost

Fuente: Elaboración propia

## Análisis de la Capacidad de Discriminación

El análisis visual ratifica los valores de AUC calculados previamente. Se observa que el modelo XGBoost (rojo) presenta la curva más externa, dominando gran parte del espacio de probabilidad con un AUC de 0.9767. Esta trayectoria indica que, matemáticamente, es el modelo con mayor capacidad de discriminación global frente a las muestras de 2026.

Por su parte, el modelo SVM (verde), con un AUC de 0.9666, muestra un comportamiento muy cercano a XGBoost, situándose como la segunda mejor opción. Es notable que, en el tramo inicial de la curva (extremo izquierdo), su capacidad de detección crece de forma muy pronunciada, superando momentáneamente a los otros modelos. Finalmente, Random Forest (azul) presenta la curva más interna de los tres (AUC de 0.9532), aunque muestra un

ligero escalonamiento, mantiene un rendimiento sobresaliente y muy alejado del clasificador aleatorio (línea diagonal).

No obstante, como se ha observado en el apartado anterior, esta superioridad en términos de AUC no se traduce necesariamente en un mejor rendimiento en el punto de operación óptimo, evidenciando diferencias en la calibración de probabilidades entre arquitecturas.

### 4.2.3 Análisis de viabilidad y operativa

Este apartado integra los hallazgos de rendimiento predictivo con las capacidades computacionales de cada arquitectura, estableciendo una comparativa final sobre su comportamiento en el escenario de 2026.

#### A. Eficiencia computacional y viabilidad en tiempo real

La viabilidad práctica de un sistema de detección de *phishing* no solo depende de su precisión, sino de su capacidad para operar sin introducir latencias perceptibles en la navegación del usuario. Para evaluar este factor, se ha diseñado un *test* de estrés consistente en 20 ejecuciones consecutivas sobre el conjunto completo de validación (1504 URLs), calculando la media de tiempo de predicción por cada unidad y su desviación estándar.

La desviación estándar refleja la variabilidad de las mediciones entre ejecuciones, permitiendo evaluar la estabilidad del tiempo de predicción.

Cabe destacar que esta medición se centra exclusivamente en la fase de inferencia del modelo, es decir, en el tiempo necesario para generar la predicción una vez que el vector de 25 atributos ha sido construido.

Los resultados se detallan en la Tabla 14:

**Tabla 14:** Evaluación de la eficiencia operativa

| Algoritmo     | Latencia Media (ms/URL) | Desviación Estándar (Std) |
|---------------|-------------------------|---------------------------|
| Random Forest | 0.0085 ms               | $\pm 0.00088$             |
| SVM           | 0.3287 ms               | $\pm 0.00951$             |
| XGBoost       | 0.0013 ms               | $\pm 0.00004$             |

Fuente: Elaboración propia

#### Análisis de Eficiencia

- **Rendimiento de los Modelos de Ensamble:** Tanto XGBoost como Random Forest demuestran una eficiencia extrema. XGBoost presenta la menor latencia, siendo capaz de procesar una URL en apenas 0.0013 milisegundos. Esta eficiencia lo convierte en el candidato ideal para despliegues en infraestructuras de red de alta disponibilidad que manejen millones de peticiones simultáneas.
- **El Coste Computacional en SVM:** El modelo SVM presenta una latencia significativamente mayor (0.32 ms), lo cual es coherente con la mayor

complejidad computacional asociada al entrenamiento y procesamiento en espacios de alta dimensionalidad mediante funciones *kernel* [12].

- **Capacidad de Procesamiento:** Para dimensionar la escalabilidad del sistema, se ha calculado la capacidad de inspección masiva de cada arquitectura. El modelo XGBoost muestra un rendimiento masivo, capaz de procesar alrededor de 770000 URLs/s, mientras que Random Forest alcanza las 117000 URLs/s. Por su parte, el modelo SVM presenta la tasa más moderada en este aspecto con aproximadamente 3000 URLs/s.

## B. Balance entre seguridad crítica y rendimiento

Al contrastar las métricas de rendimiento predictivo con los tiempos de respuesta de las tres arquitecturas, se obtiene la siguiente evaluación:

- **Eficacia Frente al Riesgo:** El modelo SVM se posiciona como la arquitectura más robusta desde la perspectiva de seguridad al ofrecer el mejor equilibrio entre errores. En un entorno de protección activa, los Falsos Negativos (FN) representan el error de mayor criticidad, ya que corresponden a amenazas que logran evadir el filtro de seguridad. Aunque XGBoost registra el menor número de Falsos Negativos (28), este resultado se obtiene a costa de un número muy elevado de Falsos Positivos (133), lo que compromete su viabilidad práctica. Por su parte, SVM mantiene un nivel de Falsos Negativos similar al de Random Forest (55 frente a 56), pero reduce drásticamente los Falsos Positivos hasta solo 12 muestras, lo que lo convierte en la opción más equilibrada y operativamente fiable.
- **Escalabilidad y Latencia:** Por el contrario, los modelos basados en ensamble demuestran una superioridad operativa clara. XGBoost, con una capacidad de procesamiento de 770000 URLs/s, se posiciona como la arquitectura más veloz.
- **Punto de Operación Óptimo:** Los resultados indican que, para el propósito de este estudio, la arquitectura SVM (umbral 0.25) ofrece el equilibrio más adecuado. Logra un *Accuracy* del 95.55% sin que su latencia (0.32 ms) suponga un obstáculo real para el usuario, validando así la arquitectura híbrida propuesta frente a las amenazas contemporáneas.

## 5 Conclusiones y trabajos futuros

### 5.1 Evaluación del cumplimiento de objetivos

El propósito fundamental de este apartado es evaluar el grado de cumplimiento de las metas planteadas al inicio de este proyecto, contrastando los objetivos planteados con los resultados finales obtenidos en la fase experimental.

#### Objetivo General

Diseñar e implementar un sistema propio basado en arquitecturas de *Machine Learning* para la detección automática de *phishing* en URLs, así como analizar sus limitaciones y proponer posibles mejoras orientadas a incrementar su robustez y aplicabilidad en entornos reales.

Grado de cumplimiento: Completado.

El objetivo general se ha alcanzado mediante el desarrollo e implementación de un motor híbrido de detección y la evaluación de tres familias algorítmicas (Random Forest, SVM y XGBoost). El sistema ha sido validado con éxito ante un escenario real y contemporáneo con muestras del año 2026, logrando identificar la configuración óptima (SVM con umbral 0.25) que equilibra la tasa de detección y la viabilidad operativa. Además, se ha cumplido con la evaluación de las limitaciones (detalladas en el apartado 5.6), las cuales sirven como base para la propuesta de mejoras técnicas orientadas a incrementar la robustez del sistema, descritas en las líneas de trabajo futuro (apartado 5.8).

#### Objetivos Específicos

- Seleccionar y caracterizar conjuntos de datos de diversas fuentes para establecer una base de entrenamiento que cubra tanto patrones clásicos como amenazas modernas.

Grado de cumplimiento: Completado

Se han seleccionado y caracterizado los conjuntos de datos antiguos (2018) y contemporáneos (2024), lo que ha permitido realizar un análisis de deriva de datos detallado en la Tabla 10. Esta selección ha servido como base para contrastar la evolución temporal del *phishing*.

- Diseñar e implementar un motor de extracción de características híbrido capaz de procesar atributos léxicos de la URL y atributos de contenido HTML en tiempo real.

Grado de cumplimiento: Completado

Se ha desarrollado con éxito un motor híbrido que transforma cada URL en un vector de 25 características numéricas y binarias, adaptadas del *dataset* de referencia PhiUSIL. El sistema procesa atributos léxicos de Nivel 1 (mediante expresiones regulares y *parsing*) y atributos HTML de Nivel 2 (mediante BeautifulSoup4). La integridad del motor fue validada mediante un test cruzado con 7335 URLs activas (*Online*), demostrando una consistencia predictiva equivalente a la de la fuente original.

- Realizar el preprocesamiento y normalización semántica de los datos para asegurar interoperabilidad entre diferentes fuentes y arquitecturas de modelos.

Grado de cumplimiento: Completado

Se ha garantizado la interoperabilidad mediante la unificación de los formatos y denominaciones del vector de 25 variables híbridas, asegurando la compatibilidad entre los conjuntos de datos antiguos (2018), contemporáneos (2024) y las muestras activas de PhishTank (2026). Este preprocesamiento e igualación semántica ha permitido que arquitecturas de distinta naturaleza operen de forma estable sobre la misma distribución.

- Implementar y comparar distintos algoritmos de clasificación supervisada para identificar la arquitectura que mejor generalice ante URLs no vistas previamente.

Grado de cumplimiento: Completado

Se han implementado y evaluado las familias Random Forest, SVM y XGBoost. El análisis comparativo permitió identificar que, si bien XGBoost lidera la capacidad de discriminación matemática teórica (AUC de 0.9767), es el algoritmo SVM el que demuestra mejor generalización práctica ante la deriva de datos, logrando una precisión muy elevada y consolidándose como la arquitectura más equilibrada desde una perspectiva operativa.

- Evaluar el rendimiento de los modelos mediante métricas de clasificación (como precisión y análisis de Falsos Positivos/ Falsos Negativos) utilizando muestras activas obtenidas en PhishTank.

Grado de cumplimiento: Completado

La evaluación externa se ejecutó utilizando un conjunto de validación independiente compuesto por muestras activas de PhishTank del año 2026. A través de un barrido de umbrales, se analizaron los balances de error reales (Falsos Positivos y Falsos Negativos), para determinar el punto de operación óptimo de cada algoritmo, obteniendo métricas representativas de rendimiento (Tabla 12) que revelan la capacidad de generalización de los modelos más allá de los resultados teóricos obtenidos en una validación interna.

- Analizar las limitaciones prácticas de los modelos obtenidos.

Grado de cumplimiento: Completado

Se han detectado y detallado las limitaciones operativas del sistema (desarrolladas en el apartado 5.6). El análisis ha permitido identificar la pérdida de eficacia si no hay acceso al código fuente de la página (restando los 10 atributos HTML), la vulnerabilidad de la inspección estática ante el contenido generado por JavaScript o técnicas de ocultación por origen, la caducidad temporal por la deriva de datos (Data

Drift) y la limitación de hardware que penaliza la portabilidad del modelo SVM frente a arquitecturas de ensamble.

- Formular y validar experimentalmente posibles mejoras orientadas a aumentar la robustez del sistema.

Grado de cumplimiento: Completado

A partir de las debilidades metodológicas y operativas detectadas en el análisis de limitaciones, se han deducido y desarrollado propuestas de mejora técnica concretas (detalladas en el apartado 5.7). Estas medidas incluyen la calibración dinámica de umbrales probabilísticos para reducir las falsas alarmas, el reentrenamiento periódico frente a la deriva de datos y el diseño de estructuras híbridas adaptativas para entornos de tráfico masivo. Estas propuestas consolidan el comportamiento del sistema y establecen la base técnica para los desarrollos a largo plazo planteados en las líneas de trabajo futuro (apartado 5.8).

## 5.2 Análisis del seguimiento de la planificación

El desarrollo cronológico del proyecto ha seguido, en líneas generales, la estructura secuencial planteada en la metodología inicial. Sin embargo, la naturaleza experimental del trabajo y la búsqueda de una coherencia técnica estricta obligaron a realizar modificaciones operativas sobre la marcha, afectando principalmente a los tiempos de la Fase 3.

A continuación, se contrasta la planificación estimada con el desarrollo real del trabajo:

- Fase 1 – Definición y planificación (Semana 1):  
Cumplida en los plazos previstos. En este período se establecieron el alcance y los objetivos del proyecto.
- Fase 2 – Revisión del estado del arte y selección de datos (Semanas 2-4):  
Esta fase se completó a tiempo en lo que respecta a la revisión bibliográfica. Aunque inicialmente se seleccionó un conjunto de datos antiguo del año 2015, los riesgos asociados a la falta de representatividad temporal no se manifestaron en esta etapa, sino al avanzar a la siguiente fase.
- Fase 3 – Desarrollo y evaluación de modelos (Semanas 5-8 Estimadas → Semanas 5-9 Reales):  
Esta fase sufrió un retraso general de una semana debido a una limitación técnica en los datos usados. Originalmente, se trabajó durante una semana en el preprocesamiento y normalización de un *dataset* de 2015. Sin embargo, al evaluar los primeros resultados, se detectó que las métricas carecían de coherencia y que era inviable construir un estudio consistente y fiel a la realidad. Ante este escenario, se planteó una reestructuración de la estrategia experimental. El objetivo pasó a ser



aplicar el motor híbrido de extracción de forma uniforme tanto sobre las muestras activas de PhishTank como sobre los *datasets* de entrenamiento, maximizando así la simetría entre variables. Dado que el *dataset* de 2015 solo contenía el vector de atributos ya extraído y carecía de las URLs en bruto, se descartó por completo, obligando a buscar una nueva fuente antigua y comenzar el proceso desde cero. Esto extendió la fase de preprocesamiento, extracción e implementación hasta la semana 9.

- Fase 4 – Análisis crítico y propuestas de mejora (Semanas 9-11 Estimadas → Semanas 10-11 Reales):

A pesar de iniciar esta fase con una semana de retraso, el tiempo dedicado a la comparativa de la brecha generacional, la deriva de datos y la comparativa de los algoritmos (SVM, Random Forest y XGBoost) fue menor de lo previsto. La unificación semántica de los *datasets* a través del motor de extracción facilitó la ejecución de las pruebas experimentales, lo que permitió absorber el retraso acumulado y finalizar el bloque experimental en la semana 11.

- Fase 5 – Redacción y revisión final (Semanas 12-13): Cumplida en los plazos previstos. La redacción de la memoria y la preparación de la entrega final se ejecutaron dentro del cronograma propuesto.

### 5.3 Evaluación del impacto ético, social y de sostenibilidad

El desarrollo de este proyecto se ha ejecutado bajo un marco de responsabilidad técnica y experimental, evaluando el impacto derivado de implantar modelos de Machine Learning en el ámbito de la ciberseguridad defensiva.

#### Consideraciones Éticas y Privacidad

El diseño del motor híbrido se ha regido por el principio de privacidad desde el diseño. Al extraer atributos basados exclusivamente en la cadena de texto de la URL y en la estructura estática del código fuente HTML, el sistema opera únicamente sobre identificadores públicos y accesibles en la red. De esta manera, se minimiza la exposición a información de carácter personal, ya que no se requiere de credenciales, formularios privados ni contenido autenticado.

Desde el punto de vista del comportamiento algorítmico, el principal dilema ético radica en el impacto operativo de los errores de clasificación:

- Falsos Positivos (12 muestras en el punto óptimo de SVM): Implican el bloqueo injustificado de sitios legítimos que comparten características técnicas con páginas fraudulentas, afectando negativamente a la experiencia del usuario y comprometiendo la confianza en el sistema de detección.
- Falsos Negativos (55 amenazas evadidas en SVM): Suponen una brecha que expone directamente al usuario al robo de credenciales.

La decisión experimental de ajustar los umbrales probabilísticos a 0.25 en el

modelo SVM responde a la responsabilidad ética de mitigar de forma óptima ambos riesgos en un entorno productivo real.

### **Impacto Social y Diversidad de Datos**

Desde la perspectiva social, este trabajo analiza el rendimiento de herramientas automáticas basadas en aprendizaje automático frente al fraude digital, un problema condicionado actualmente por la sofisticación de los ataques y la automatización en la generación de páginas fraudulentas. El análisis expone que el uso de estos sistemas automatizados conlleva el riesgo de generar una excesiva dependencia en el usuario final, lo que podría reducir su capacidad crítica y su nivel de alerta ante ataques de ingeniería social que consigan evadir los filtros algorítmicos. Por tanto, el estudio constata que estos modelos no deben sustituir a la concienciación humana, sino plantearse como una capa complementaria de control.

En cuanto al sesgo y la diversidad de los datos, el proyecto ha abordado esta cuestión desde tres enfoques principales: diversidad temporal, diversidad de fuente y equilibrio entre clases. La diversidad temporal se ha trabajado comparando un *dataset* antiguo de 2018, un *dataset* contemporáneo de 2024 y muestras activas de PhishTank de 2026, lo que ha permitido comprobar cómo evolucionan los patrones de phishing. La diversidad de fuente se ha reforzado combinando URLs fraudulentas procedentes de repositorios y plataformas colaborativas con dominios legítimos de alta reputación, permitiendo evaluar tanto la detección de amenazas como el riesgo de Falsos Positivos. Finalmente, el balanceo entre muestras de *phishing* y legítimas ha buscado reducir el sesgo hacia una clase mayoritaria, garantizando que el sistema aprenda a discriminar ambas categorías bajo condiciones experimentales equitativas. De este modo, la diversidad de datos no se entiende únicamente como volumen, sino como representatividad suficiente para evaluar la robustez del modelo ante distintos contextos temporales y operativos.

### **Sostenibilidad Computacional**

A diferencia de los modelos de aprendizaje profundo (*Deep Learning*) que requieren hardware de alto rendimiento y un consumo energético elevado, en este estudio se ha priorizado el uso de algoritmos de aprendizaje supervisado (Random Forest, SVM y XGBoost). Esta decisión técnica permite que tanto el proceso de preprocesamiento de datos como el entrenamiento y validación de los modelos puedan ejecutarse íntegramente en un ordenador convencional en local, minimizando el consumo de recursos computacionales.

La viabilidad de este enfoque para entornos con recursos limitados se confirma mediante los resultados reales de latencia obtenidos en las pruebas (Tabla 14):

- Los modelos basados en árboles de decisión, especialmente XGBoost, demuestran una eficiencia muy alta en la fase de explotación con un tiempo de procesamiento de apenas 0.0013 ms por URL (permitiendo evaluar unas 770000 URL/s).
- El modelo SVM, seleccionado por su precisión global, se sitúa en 0.32 ms por URL (~3000 URLs/s) debido a la carga matricial que requiere el escalado de datos.

Estos tiempos demuestran que el sistema es computacionalmente ligero y eficiente, lo que garantiza que su despliegue operativo en producción no requiere de servidores de gran capacidad ni infraestructuras que disparen el consumo eléctrico. Esto refuerza su viabilidad como solución sostenible y operativamente accesible.

## 5.4 Conclusiones del Estudio de Obsolescencia

El análisis de obsolescencia realizado en este trabajo constituye una prueba empírica de que la ciberseguridad basada en *Machine Learning* no es un proceso estático, sino un entorno en el que la ofensiva y la defensa avanzan de forma recíproca acelerada. Tras confrontar el modelo antiguo (2018) y el contemporáneo (2024) contra la realidad del año 2026, se extraen las siguientes conclusiones:

### El Colapso de las Métricas de Laboratorio y el Impacto Real en la Seguridad Organizacional

Una de las conclusiones más reveladoras es el peligro del exceso de confianza técnica basada exclusivamente en métricas internas. Mientras que ambos modelos muestran un rendimiento casi perfecto en “laboratorio”, la validación externa en sus puntos de operación óptimos revela la verdadera brecha de seguridad. Aunque la diferencia en *Accuracy* (9.10%) pueda parecer moderada sobre el papel, el análisis de los Falsos Negativos (FN) desvela una vulnerabilidad crítica: el modelo de 2018 deja pasar un 63% más de amenazas que el modelo contemporáneo (62 frente a 38). Para dimensionar este riesgo, si consideramos que una organización pequeña o mediana puede recibir cientos de intentos de *phishing* mensuales, una diferencia de 24 amenazas no detectadas es inasumible. En el ecosistema de ciberseguridad actual, un solo Falso Negativo que logre evadir el filtro puede traducirse potencialmente en una brecha de datos masiva o en un incidente de *ransomware*, lo que concluye que el modelo antiguo no es solo menos preciso, sino operativamente más peligroso para la continuidad del negocio.

### El Falso “Candado de Seguridad”: La Obsolescencia del Protocolo HTTPS

Los datos obtenidos mediante el análisis de deriva de datos permiten concluir que el *phishing* ha “profesionalizado” su infraestructura de forma drástica. El crecimiento exponencial del 3180% en el uso de certificados HTTPS en muestras maliciosas representa un cambio radical respecto a los modelos de seguridad previos. Lo que en 2018 parecía un descriptor fiable de legitimidad, en la actualidad es una táctica de ofuscación estándar para generar una falsa sensación de seguridad. Esta conclusión invalida cualquier sistema de detección que siga otorgando un peso positivo por defecto a la presencia de SSL, confirmando que el cifrado se ha convertido en una herramienta de ingeniería social diseñada específicamente para proyectar un falso candado de seguridad y anular la precaución del usuario ante sitios fraudulentos.

### Evolución Hacia la Estética Minimalista y Complejidad de Infraestructura

A través del estudio léxico se concluye que existe una tendencia clara hacia la “limpieza visual” de las URLs fraudulentas. La reducción en la longitud de las direcciones (-39.64%) y la eliminación de caracteres ruidosos como *ampersands*

“&” (-87%) o igualdades “=” (-80%) indican que el atacante busca mimetizarse con la estética simplificada de las aplicaciones web modernas. Sin embargo, esta simplificación visual se compensa con un aumento en la complejidad técnica de la arquitectura de dominios, donde el incremento del 92% en el uso de subdominios revela que el fraude se oculta ahora en capas de infraestructura más profundas, menos evidentes para el ojo humano pero detectables mediante el análisis de los 25 atributos híbridos propuestos en este proyecto.

### **Reorganización de la Jerarquía de Importancia de los Atributos**

La comparativa de la importancia relativa de las variables entre 2018 y 2024 revela un cambio profundo en los indicadores que el modelo utiliza para discriminar el fraude. No se observa una prevalencia de un nivel sobre otro (Léxico vs. HTML), sino una evolución dispar en la capacidad predictiva de atributos específicos:

- **Pérdida de Relevancia de los Metadatos y la Estructura de Red:** El hallazgo más notable es la caída de *HasDescription\_Extracted* (HTML), que pasa de ser el predictor principal en 2018 a perder más del 50% de su peso en el modelo actual. De igual forma, indicadores de red y estructura de la URL que antes eran determinantes, como *TLDLength*, *NoOfSubDomain* o *DomainLength*, han visto reducida su importancia a niveles mínimos en el modelo contemporáneo. Este comportamiento resulta especialmente significativo en el caso de los atributos como *NoOfSubDomain* y *DomainLength*, que pese a haber incrementado su frecuencia en las muestras contemporáneas, han perdido capacidad discriminativa dentro del proceso de decisión del modelo.
- **Consolidación de Indicadores de Infraestructura y Protocolo:** Por el contrario, el modelo de 2024 apoya su capacidad de decisión en dos pilares fundamentales: la arquitectura de contenido mediante *NoOfiFrame\_Extracted* (HTML) y la seguridad del canal a través de *IsHTTPS* (Léxico). Ambas variables han experimentado el mayor crecimiento, convirtiéndose en los nuevos referentes de peso para la clasificación.
- **Subida en Patrones Léxicos de Ofuscación:** Atributos relacionados con la composición anómala de la URL, como *NoOfOtherSpecialCharsInURL*, *DegitRatioInURL* o *NoOfDegitsInURL*, han ganado una presencia relevante que no tenían en el modelo antiguo.

### **La Actualización Continua Como Paradigma de Defensa**

En definitiva, los resultados del estudio de obsolescencia permiten concluir que la eficacia de un sistema de detección de *phishing* no reside solamente en la complejidad de su algoritmo, sino en la vigencia cronológica de su base de conocimiento. La transferencia de importancia desde patrones descriptivos hacia indicadores de infraestructura técnica demuestra que los atacantes han asimilado y neutralizado los criterios de seguridad de la década pasada. Por tanto, se concluye que cualquier solución de detección basada en modelos estáticos está condenada a la disfuncionalidad operativa en cuestión de pocos años. Este hallazgo valida la estrategia metodológica de este trabajo y subraya que, en el escenario actual de la ciberseguridad, la recalibración constante de los pesos de las variables y la incorporación de muestras activas no son tareas

de mantenimiento, sino requisitos estructurales para garantizar la robustez del sistema frente a las amenazas del futuro. En este sentido, los resultados obtenidos confirman el objetivo central del proyecto, evidenciando que la capacidad de generalización del modelo y su aplicabilidad en entornos reales dependen directamente de su adaptación continua a patrones contemporáneos, y no únicamente de su rendimiento en escenarios de laboratorio.

## **5.5 Conclusiones del Análisis de Algoritmos**

Este bloque finaliza el estudio técnico comparando la eficacia de las tres familias matemáticas seleccionadas (Random Forest, SVM y XGBoost) ante el vector de 25 atributos híbridos. Una vez descartadas las métricas de laboratorio por su falta de representatividad, se extraen las siguientes conclusiones sobre el rendimiento real de los modelos en el escenario de 2026:

### **El Contraste Entre el Rendimiento Teórico y el Desempeño Real**

Una conclusión fundamental de este análisis es que el Área bajo la curva (AUC) no es un indicador absoluto de éxito en entornos de seguridad crítica. Matemáticamente, XGBoost lidera la capacidad de discriminación con un AUC de 0.9767, seguido de cerca por SVM (0.9666) y Random Forest (0.9532). Sin embargo, al trasladar este potencial a un punto de operación real, se observa que la superioridad matemática de XGBoost se traduce en una excesiva sensibilidad: genera 133 Falsos Positivos (falsas alarmas), una cifra inasumible para la experiencia de usuario. Por el contrario, el modelo SVM demuestra la mayor robustez práctica, logrando el mejor equilibrio entre detección y usabilidad. Los resultados evidencian que la eficacia operativa no depende solo del AUC, sino de la estabilidad del modelo en su punto de operación óptimo.

### **SVM Como Arquitectura de Referencia Para la Seguridad Equilibrada**

Al evaluar los modelos bajo el prisma de la viabilidad operativa, el algoritmo SVM (umbral 0.25) se consolida como la arquitectura más fiable. Su principal fortaleza reside en la minimización de los Falsos Positivos, logrando reducir las falsas alarmas a tan solo 12 muestras, frente a las 24 de Random Forest y las 133 de XGBoost.

En un entorno organizacional, un sistema de detección que bloquea por error una cantidad elevada de sitios legítimos genera una degradación de la productividad y una desconfianza del usuario en las herramientas de seguridad. Se concluye que la lógica de los vectores de soporte (SVM) ofrece la mejor generalización ante la deriva de datos, logrando una precisión muy elevada (0.98). Cabe destacar que, en cuanto a la tasa de Falsos Negativos (ataques escapados), el modelo Random Forest sigue muy de cerca al SVM con 56 frente a 55 muestras, demostrando ambos una consistencia superior a la de XGBoost en este punto de operación. Aunque XGBoost es el modelo que menos ataques deja escapar (28 FN), su incapacidad para distinguir correctamente el tráfico legítimo en el escenario de 2026 sitúa al SVM, con el respaldo inmediato de Random Forest, como la solución más consistente y profesional.

### **Eficiencia Computacional y Escalabilidad del Despliegue**

El análisis de latencias confirma que las arquitecturas seleccionadas garantizan una integración fluida en entornos de producción. Existe una diferencia de escala notable entre algoritmos, donde XGBoost alcanza velocidades masivas de unas 770000 URLs/s, seguido por la notable eficiencia de Random Forest con aproximadamente 117000 URLs/s. Por su parte, la latencia media de SVM (0.32 ms por URL) es despreciable en comparación con los tiempos de carga habituales de una web. Estos resultados concluyen que el sistema es apto para su despliegue en aplicaciones web o extensiones de navegador sin degradar la experiencia de usuario. Incluso bajo picos de tráfico intenso, la capacidad de procesar alrededor de 3000 URLs/s asegura que el motor de detección no se convierta en un cuello de botella para la infraestructura de red.

### **Conclusión del Balance Técnico-Operativo**

En definitiva, los resultados validan la configuración SVM con umbral 0.25 como la solución óptima para el motor de detección híbrido desarrollado. Mientras que XGBoost ofrece una potencia de cálculo superior y Random Forest se presenta como un estándar muy equilibrado, SVM es el único que proporciona el equilibrio necesario entre la detección de amenazas y la minimización de falsas alarmas. Se concluye que, ante la sofisticación del *phishing* contemporáneo, la capacidad de SVM para definir hiperplanos de separación óptimos demostró un comportamiento más robusto dentro del contexto experimental desarrollado que los modelos basados en ensamble (*Bagging* y *Boosting*), estableciendo la base necesaria para un despliegue de seguridad confiable y eficiente.

No obstante, es necesario señalar que la elección de la arquitectura final depende directamente del volumen de tráfico del entorno de despliegue. Si bien SVM ofrece la mayor robustez en términos de equilibrio entre errores, en escenarios de tráfico masivo o infraestructuras de alta disponibilidad donde una capacidad de ~3,000 URLs/s pudiera resultar limitada, el modelo Random Forest (umbral 0.10) se consolida como una alternativa altamente viable. Al ofrecer un compromiso sólido entre capacidad de detección (56 FN) y una velocidad de procesamiento significativamente superior (117,000 URLs/s), Random Forest permite absorber picos de carga sin degradar el rendimiento del sistema, posicionándose como la opción más adecuada en entornos de inspección a gran escala.

## **5.6 Análisis de Limitaciones**

A pesar de la solidez técnica demostrada por la arquitectura propuesta, el desempeño del sistema de detección está sujeto a una serie de limitaciones y factores externos que deben ser considerados para su despliegue en entornos reales.

### **Dependencia de la Disponibilidad de Recursos HTML**

La naturaleza híbrida del vector de 25 atributos constituye tanto una fortaleza como una limitación operativa. El sistema requiere acceso al código fuente de la página y a elementos del DOM (Document Object Model), como *iFrames*,

campos de formulario o metadatos, para alcanzar los niveles de precisión documentados.

- **Limitación:** En escenarios donde la URL es analizada de forma aislada (por ejemplo, en servicios de resolución de DNS), los 10 atributos de nivel HTML no estarían disponibles.
- **Impacto:** Una degradación en la disponibilidad de estos atributos obligaría al sistema a operar exclusivamente con los 15 descriptores léxicos, lo que previsiblemente aumentaría la tasa de Falsos Negativos al eliminar señales estructurales clave utilizadas por el modelo para discriminar páginas fraudulentas.

### **Vulnerabilidad Ante el Contenido Dinámico y Técnicas de Ocultación**

El análisis de contenido basado en la extracción automatizada de datos estáticos presenta desafíos frente a la evolución de las contramedidas empleadas por los atacantes:

- **Contenido Generado por JS:** Dado que el presente modelo realiza una inspección de tipo estático, la renderización de contenido mediante *JavaScript* en el lado del cliente se define como una limitación de alcance. Si los atributos HTML se generan dinámicamente tras la ejecución de scripts, el análisis podría basarse en una estructura preliminar que no contenga la totalidad de los elementos maliciosos.
- **Técnicas de Evasión por Detección de Origen:** Algunos servidores maliciosos emplean filtros para identificar la identidad del visitante. Si el servidor detecta que la consulta proviene de una herramienta de seguridad o un centro de datos, entrega contenido legítimo, reservando el fraude exclusivamente para el usuario final. Este fenómeno representa un límite estructural para cualquier sistema de detección basado en análisis estático, al introducir un comportamiento adaptativo que escapa a la observación *offline* del modelo.

### **El Factor Temporal y el Ciclo de Vida del *Phishing***

Los resultados del estudio de obsolescencia han demostrado que los patrones de fraude tienen una vigencia limitada.

- **Deriva de Datos:** Aunque el modelo de 2024 ha mostrado una excelente generalización frente a muestras de 2026, la creciente sofisticación en la creación de URLs que buscan imitar con mayor precisión patrones de legitimidad mediante técnicas automatizadas, podría degradar la eficacia de la SVM actual al desdibujar la frontera entre las clases.
- **Limitación de la Base de Conocimiento:** El sistema es tan robusto como lo sea su último entrenamiento. La falta de un mecanismo de reentrenamiento automatizado o de un pipeline de datos continuo constituye una limitación para mantener la vigencia del sistema a largo plazo.

### **Dependencia de la Calidad y Representatividad del Conjunto de Datos**

El rendimiento del sistema está condicionado por la naturaleza de los datos utilizados en el entrenamiento y la validación. El uso de plataformas como PhishTank introduce limitaciones inherentes que deben ser consideradas:

- **Sesgo de Recogida:** Las URLs reportadas en plataformas colaborativas pueden no representar la totalidad del ecosistema de fraude, ya que dependen de la detección previa por parte de usuarios o sistemas externos. Esto implica que el modelo es experto en detectar patrones ya conocidos por la comunidad, pero podría presentar dificultades ante ataques dirigidos de baja visibilidad (*spear phishing*).
- **Desbalance Temporal:** Aunque se ha validado con muestras de 2026, existe el riesgo de que ciertos vectores emergentes, cuya presencia en los repositorios de acceso público es todavía escasa, no estén suficientemente representados en los datos de entrenamiento.
- **Distribución Balanceada de Clases:** El entrenamiento y validación se realizaron sobre conjuntos balanceados (50% *phishing* y 50% legítimo) para facilitar una comparación homogénea entre modelos. No obstante, esta distribución no representa el tráfico real, donde las URLs legítimas son ampliamente mayoritarias. En un entorno real, este desbalance podría afectar especialmente a la tasa de Falsos Positivos.
- **Generalización Geográfica y Contextual:** Los patrones de *phishing* varían según el idioma, la región o el tipo de servicio suplantado. La capacidad del modelo para generalizar fuera del dominio observado podría verse afectada ante campañas diseñadas para nichos de mercado o regiones geográficas con comportamientos digitales específicos.

En conjunto, estos factores indican que, si bien los resultados son excelentes en el marco del estudio, el rendimiento operativo podría experimentar variaciones al enfrentarse a distribuciones de datos que no sigan los criterios de las fuentes públicas utilizadas.

### Limitaciones de Hardware en el Despliegue de SVM

Como se detalló en el análisis de viabilidad, la arquitectura SVM presenta una carga computacional superior debido a la necesidad de preprocesamiento (escalado de datos) y al cálculo matricial de los vectores de soporte.

- **Condicionante de Hardware:** Mientras que los modelos de ensamble podrían ejecutarse en dispositivos con recursos muy limitados, el modelo SVM requiere una mayor capacidad de cómputo relativa para mantener latencias óptimas, lo que limita su portabilidad en comparación con Random Forest.

### Limitación en la Aplicabilidad Directa en Entornos Críticos

A pesar del notable rendimiento global observado, el sistema presenta limitaciones relevantes para su despliegue como solución única en entornos productivos. En el punto de operación óptimo del modelo SVM, se registran 55 Falsos Negativos sobre un total de 752 muestras de *phishing*, lo que implica que aproximadamente un 7% de las amenazas logran evadir el filtro.

Desde una perspectiva puramente estadística, un nivel de *Recall* del 92.68% se considera notable. Sin embargo, en el contexto de la ciberseguridad, donde el coste de un único fallo puede derivar en el compromiso de credenciales críticas, brechas de datos masivas o incidentes de *ransomware*, esta tasa de error residual resulta significativa. Un sistema que deja pasar 7 de cada 100 ataques



no puede ser la única barrera de defensa en una infraestructura organizacional sensible.

Esta limitación pone de manifiesto que el sistema propuesto, en su estado actual, no debe ser considerado como un mecanismo de protección autónomo. Su función óptima es la de actuar como una capa adicional dentro de una arquitectura de defensa en profundidad, debiendo ser complementado con otras técnicas de seguridad como el análisis de reputación de dominios, la monitorización de tráfico en tiempo real y, fundamentalmente, la concienciación del usuario final.

Por tanto, se concluye que, aunque el modelo demuestra una alta capacidad de discriminación, su aplicación en entornos reales debe integrarse en un ecosistema de seguridad multicapa que permita mitigar el impacto de los ataques que logran superar la detección algorítmica.

## **5.7 Propuestas de mejora y robustez del sistema**

A partir de los resultados experimentales obtenidos durante el desarrollo del proyecto, se han identificado diferentes ideas orientadas a incrementar la robustez, estabilidad y capacidad de generalización del sistema frente a escenarios reales de *phishing* contemporáneo.

### **Optimización Dinámica de Umbrales**

Uno de los hallazgos más relevantes del estudio ha sido el impacto directo que tiene el ajuste de los umbrales probabilísticos sobre el equilibrio entre Falsos Positivos y Falsos Negativos. Los resultados demostraron que el comportamiento de los modelos varía significativamente dependiendo del punto de operación seleccionado.

En este contexto, una posible mejora consistiría en implementar un sistema de ajuste dinámico de umbrales en función del entorno operativo. En escenarios corporativos críticos, donde minimizar amenazas resulta prioritario, el sistema podría operar con umbrales más agresivos orientados a minimizar los Falsos Negativos y maximizar la capacidad de detección de amenazas. Por el contrario, en entornos orientados a usuario final o navegación general, podría priorizarse la reducción de falsas alarmas para mejorar la experiencia de uso.

### **Actualización Continua y Reentrenamiento Periódico**

El análisis de deriva de datos realizado entre los *datasets* de 2018, 2024 y las muestras activas de 2026 evidenció que los patrones de *phishing* evolucionan constantemente. Variables anteriormente relevantes, como el uso de HTTP o determinados caracteres especiales en la URL, han perdido gran parte de su capacidad discriminativa debido a la profesionalización de las campañas actuales.

A partir de esta observación, se propone incorporar mecanismos de reentrenamiento periódico utilizando muestras recientes obtenidas de plataformas colaborativas, como el propio PhishTank, o fuentes especializadas.

Esta estrategia permitiría mantener actualizado el espacio de representación del modelo y reducir progresivamente la degradación provocada por la deriva de datos.

### **Ampliación del Vector de Características**

Otra posible línea de mejora consistiría en ampliar el vector híbrido de atributos utilizado por el motor de extracción. Aunque el sistema actual ha demostrado una capacidad de generalización elevada utilizando 25 variables léxicas y estructurales, los resultados obtenidos durante el análisis de deriva de datos evidencian que los patrones de phishing evolucionan continuamente, modificando la relevancia de determinados indicadores con el paso del tiempo.

Por ello, resultaría interesante incorporar nuevos atributos relacionados con reputación de dominios, antigüedad del certificado TLS, análisis de redirecciones complejas, comportamiento temporal del dominio o características derivadas del contenido visual de la página. Del mismo modo, podrían añadirse variables orientadas a detectar técnicas modernas de evasión basadas en acortadores de enlaces o generación automatizada de subdominios.

La incorporación progresiva de nuevas variables permitiría enriquecer el espacio de representación del modelo y aumentar su capacidad de adaptación frente a amenazas de *phishing* cada vez más sofisticadas.

### **Arquitecturas Híbridas Adaptativas**

Los resultados experimentales muestran que no existe una arquitectura universalmente superior en todos los escenarios. Mientras que SVM ofrece el mejor equilibrio global entre capacidad de detección, precisión y estabilidad operativa, XGBoost presenta una capacidad de procesamiento masivo muy superior y Random Forest demuestra una alta robustez general.

En este sentido, una posible línea de mejora consistiría en desarrollar una arquitectura híbrida adaptativa capaz de combinar múltiples modelos según el contexto de ejecución. Por ejemplo, Random Forest o XGBoost podrían utilizarse como filtros preliminares de alta velocidad en infraestructuras con tráfico masivo, reservando SVM para etapas posteriores de verificación de amenazas más críticas.

### **Análisis de Sensibilidad al Ruido y Calidad de Datos**

Durante la fase experimental se observó un comportamiento relevante en el modelo Random Forest. En el análisis de obsolescencia, entrenado sobre un conjunto de datos más reducido y controlado, el modelo alcanzó un *Accuracy* superior al obtenido posteriormente durante la comparativa de algoritmos utilizando un conjunto de datos más amplio.

Este comportamiento podría sugerir que el incremento del volumen y diversidad de las muestras introduce niveles adicionales de ruido estadístico que afectan negativamente a la precisión del modelo. Por ello, resultaría de interés realizar futuros estudios centrados en la sensibilidad de las arquitecturas frente al ruido, evaluando técnicas de selección de características, depuración de muestras

conflictivas o reducción de dimensionalidad orientadas a maximizar la estabilidad predictiva.

### **Integración Multicapa en Sistemas Reales**

Finalmente, los resultados obtenidos indican que el sistema desarrollado puede integrarse como una capa adicional dentro de arquitecturas de defensa en profundidad. En lugar de plantearse como un mecanismo aislado, el modelo podría combinarse con sistemas de reputación de dominios, análisis de tráfico de red, motores SIEM o herramientas de monitorización.

Esta aproximación multicapa permitiría compensar las limitaciones del análisis estático y aumentar significativamente la resiliencia del sistema frente a amenazas de fraude digital avanzadas.

## **5.8 Líneas de trabajo futuro**

El desarrollo de este proyecto establece una base experimental sólida para la detección automatizada de phishing mediante *Machine Learning*. No obstante, la evolución constante de las técnicas de fraude digital y las limitaciones identificadas durante el estudio permiten proyectar diversas líneas de investigación y desarrollo tecnológico orientadas a ampliar la robustez, capacidad de adaptación y viabilidad operativa del sistema a largo plazo.

### **Automatización del Ciclo de Vida del *Dataset***

Una de las principales líneas de continuidad del proyecto consiste en la construcción de una infraestructura automatizada para la generación periódica de conjuntos de datos actualizados. Los resultados obtenidos durante el estudio de obsolescencia han demostrado que la deriva temporal modifica de forma significativa la relevancia de los atributos y degrada progresivamente la eficacia de los modelos estáticos.

En este contexto, se plantea el desarrollo de un *pipeline* automatizado capaz de ejecutar, en intervalos regulares de 3, 6 o 12 meses, tareas de recolección, filtrado, extracción de características y balanceo de clases utilizando muestras activas procedentes de plataformas como PhishTank, junto con dominios legítimos de alta reputación.

La automatización de este proceso permitiría consolidar un *dataset* propio y continuamente actualizado, garantizando la vigencia cronológica de los modelos y reduciendo la dependencia de *datasets* públicos estáticos publicados por terceros.

### **Desarrollo de un Motor de Extracción Avanzado**

El motor híbrido implementado en este trabajo ha demostrado una elevada capacidad de generalización utilizando 25 atributos léxicos y estructurales. Sin embargo, su funcionamiento continúa dependiendo parcialmente de heurísticas de *parsing* estático y librerías de terceros, lo que introduce ciertas limitaciones en términos de precisión, flexibilidad y eficiencia operativa.

Como línea futura, se plantea el desarrollo de un motor de extracción nativo optimizado a bajo nivel, orientado a maximizar el rendimiento y reducir la latencia de explotación en entornos de alto tráfico. Esta evolución permitiría minimizar la dependencia de heurísticas manuales y aproximaciones estáticas, favoreciendo la obtención de representaciones más precisas, consistentes y cercanas al comportamiento real de las muestras analizadas. Del mismo modo, futuras versiones del motor podrían ampliar significativamente el espacio de representación del sistema, superando los 25 atributos actuales y mejorando la precisión en la captura y procesamiento de metadatos, manteniendo al mismo tiempo una alta eficiencia computacional.

### **Despliegue en Producción Mediante Arquitecturas de Servicios**

Con el objetivo de trasladar el sistema desarrollado desde un entorno experimental hacia escenarios operativos reales, una posible línea de trabajo futuro consistiría en desplegar el motor híbrido de detección y la arquitectura SVM optimizada mediante servicios desacoplados y contenedores virtualizados.

Esta aproximación permitiría exponer las capacidades de clasificación del modelo a través de interfaces ligeras de comunicación, facilitando su integración con aplicaciones web, infraestructuras de red o sistemas de seguridad perimetral. De este modo, el sistema podría evaluar automáticamente URLs en tiempo real y devolver tanto la clasificación final como indicadores probabilísticos asociados al nivel de riesgo detectado.

### **Integración en Arquitecturas de Defensa Multicapa**

Otra línea de trabajo futuro consistiría en integrar el sistema desarrollado dentro de arquitecturas de defensa en profundidad orientadas a la detección y mitigación de amenazas web.

Dado que los resultados experimentales evidencian que ningún modelo alcanza una capacidad de detección absoluta, el sistema desarrollado se podría utilizar como una capa complementaria junto a mecanismos tradicionales de seguridad, como sistemas de reputación de dominios, monitorización de tráfico, análisis de comportamiento o filtrado perimetral.

Esta aproximación permitiría combinar las capacidades predictivas del modelo con otras fuentes de contexto operativo, aumentando la resiliencia global del sistema frente a campañas de *phishing* avanzadas y reduciendo el impacto de los ataques que logran evadir una única capa de protección.

## 6 Glosario

- **Accuracy (Exactitud):** Métrica de evaluación que representa el porcentaje total de predicciones correctas (tanto positivas como negativas) realizadas por un modelo de clasificación sobre el total de muestras analizadas.
- **Ataque homógrafo:** Técnica de fraude que consiste en registrar un nombre de dominio visualmente casi idéntico al de un sitio web legítimo, sustituyendo caracteres estándar por otros pertenecientes a alfabetos diferentes para engañar al usuario.
- **Atributo HTML (atributo de contenido):** Característica predictiva extraída directamente del código fuente de la página web utilizada para detectar indicadores de fraude estructural.
- **Atributo léxico:** Característica predictiva obtenida mediante el análisis textual y estadístico de la cadena de caracteres que compone una URL (como su longitud o número de puntos).
- **AUC (Area Under the Curve):** Métrica que cuantifica el área bajo la curva ROC. Mide la capacidad global de un modelo para clasificar correctamente las muestras positivas y negativas a lo largo de todos los umbrales de decisión posibles.
- **Bagging:** Técnica de aprendizaje por ensamble que entrena múltiples clasificadores independientes en paralelo sobre diferentes subconjuntos aleatorios de datos, promediando sus predicciones para reducir la varianza general del sistema.
- **Balanceo de clases:** Proceso de preprocesamiento técnico enfocado en igualar la proporción de muestras legítimas y de *phishing* en el conjunto de datos, evitando que los algoritmos sufran sesgos hacia la clase mayoritaria.
- **Boosting:** Técnica de aprendizaje por ensamble que entrena modelos de forma secuencial, donde cada nuevo clasificador se diseña específicamente para corregir los errores de predicción cometidos por los modelos anteriores.
- **Clasificación binaria:** Tarea de aprendizaje supervisado en la que el modelo debe asignar cada muestra de entrada a una de dos categorías o clases excluyentes.
- **Curva ROC (Receiver Operating Characteristic):** Gráfico que representa el rendimiento de un modelo de clasificación en todos los umbrales de decisión, contrastando la tasa de verdaderos positivos frente a la tasa de falsos positivos.
- **Data Drift (Deriva de datos):** Variación estadística que sufren los patrones y la distribución de las variables con el paso del tiempo, lo que provoca la obsolescencia cronológica de los modelos fijos.
- **Dataset (Conjunto de datos):** Colección organizada y estructurada de registros y variables que sirve como base de conocimiento para entrenar, validar y probar las arquitecturas de *Machine Learning*.
- **Desviación estándar:** Medida estadística que cuantifica el grado de dispersión o variabilidad de un conjunto de datos respecto a su valor medio.

- **DOM (*Document Object Model*):** Estructura jerárquica en forma de árbol que representa los elementos de una página web tras ser procesada por el navegador, permitiendo la inspección dinámica de su contenido.
- **Ensamble:** Enfoque de *Machine Learning* que consiste en combinar las predicciones de múltiples modelos individuales para generar una predicción única.
- **Falso Negativo (FN):** Error de seguridad crítico que se produce cuando el modelo clasifica erróneamente una URL de *phishing* como un sitio web legítimo, permitiendo que la amenaza evada el filtro.
- **Falso Positivo (FP):** Error que ocurre cuando el modelo etiqueta una página web legítima como si fuera *phishing*, generando una falsa alarma que bloquea el acceso al usuario.
- **F1-Score:** Métrica de evaluación que combina la precisión y el *recall* en un único indicador, ofreciendo un balance técnico del rendimiento del modelo.
- **Heurísticas:** Criterios lógicos predefinidos que se utilizan para resolver un problema de forma rápida o extraer características concretas sin necesidad de recurrir a un procesamiento estadístico complejo.
- **Hiperplano:** Límite geométrico de decisión calculado por el algoritmo SVM en un espacio multidimensional para separar de forma óptima las muestras pertenecientes a diferentes clases.
- **HTML (*Hypertext Markup Language*):** Lenguaje de marcado estándar utilizado para la construcción, estructuración y definición del contenido de las páginas web en Internet.
- **HTTPS (*Hypertext Transfer Protocol Secure*):** Versión segura y cifrada del protocolo de transferencia web que utiliza certificados SSL/TLS para proteger la integridad y confidencialidad del canal de comunicación.
- **Ingeniería social:** Conjunto de técnicas psicológicas y de manipulación que emplean los atacantes para engañar a las personas y conseguir que revelen información confidencial o realicen acciones perjudiciales.
- **Latencia:** Intervalo de tiempo computacional que requiere el motor de extracción y el modelo entrenado para procesar una URL y devolver una respuesta predictiva.
- **Machine Learning:** Rama de la inteligencia artificial centrada en el desarrollo de algoritmos capaces de identificar patrones complejos a partir de datos para aprender a realizar tareas de forma automatizada.
- **Matriz de confusión:** Tabla bidimensional de contingencia que desglosa el rendimiento real de un modelo de clasificación al cruzar sus predicciones (positivas y negativas) con los valores verdaderos del *dataset*.
- **Motor de extracción híbrido:** Módulo de software desarrollado en este proyecto programado para capturar y unificar en un vector numérico tanto los atributos textuales de la URL como los estructurales de su HTML.
- **Normalización semántica:** Proceso de limpieza técnica encargado de homogeneizar los formatos, nombres, escalas y tipos de variables procedentes de distintas bases de datos para garantizar su interoperabilidad.
- **Overfitting (*Sobreajuste*):** Problema que surge cuando un modelo se adapta de manera excesiva a los datos de entrenamiento, memorizando

el ruido estadístico y perdiendo su capacidad para generalizar ante URLs nuevas.

- **Parsing:** Proceso de análisis sintáctico y extracción de datos mediante el cual una secuencia de texto en bruto (como una URL) se descompone y se transforma en estructuras de datos manejables.
- **Phishing:** Modalidad de ciberataque basada en la suplantación de identidad de marcas o instituciones de confianza con el objetivo de engañar al usuario para robarle credenciales y datos financieros.
- **PhishTank:** Plataforma comunitaria global y base de datos colaborativa que recopila, verifica y distribuye listados actualizados de URLs fraudulentas reportadas por analistas y usuarios.
- **PhiUSIIL:** *Dataset* académico que recopila miles de instancias web con atributos léxicos y de contenido, utilizado en este trabajo para el entrenamiento del sistema.
- **Pipeline:** Cadena estructurada de procesos de software conectados secuencialmente donde la salida de cada etapa sirve como entrada para la siguiente.
- **Precision (Precisión):** Métrica de evaluación que mide la proporción de sitios identificados como *phishing* que realmente corresponden a ataques reales, reflejando la fiabilidad del modelo ante falsas alarmas.
- **Preprocesamiento:** Etapa inicial de limpieza y adecuación de los datos en bruto para convertirlos en un formato apto antes de introducirlos en los modelos de *Machine Learning*.
- **Punycode:** Sistema de codificación utilizado para traducir cadenas de caracteres Unicode (como los caracteres con tildes o alfabetos no latinos) al formato ASCII estándar.
- **Random Forest (Bosque aleatorio):** Algoritmo de aprendizaje supervisado basado en ensamble que combina las predicciones de múltiples árboles de decisión independientes para generar una predicción más robusta y estable.
- **Recall:** Métrica de evaluación que calcula la proporción de ataques reales de *phishing* que el modelo es capaz de detectar sobre el total de amenazas existentes, midiendo su nivel de seguridad.
- **Spear Phishing:** Variante de *phishing* dirigida y personalizada orientada a un individuo, empresa o perfil específico, empleando información del contexto de la víctima para aumentar su éxito.
- **SSL/TLS:** Protocolos criptográficos diseñados para establecer conexiones seguras y cifradas en redes de ordenadores, garantizando la autenticidad e integridad de la transferencia de datos.
- **SVM (Support Vector Machine/ Máquina de Vectores de Soporte):** Algoritmo de clasificación supervisada cuyo objetivo es encontrar el hiperplano óptimo de separación maximizando la distancia (margen) entre las muestras de clases distintas más cercanas.
- **Umbral de decisión:** Límite probabilístico ajustable (por defecto en 0.50) que determina la frontera a partir de la cual el clasificador asigna una muestra a la clase legítima o a la clase de *phishing*.
- **URL (Uniform Resource Locator):** Cadena de caracteres única y estandarizada que se utiliza para direccionar, identificar y localizar un recurso específico en la red de Internet.

- **Validación externa:** Prueba de robustez que consiste en evaluar el rendimiento de los modelos entrenados utilizando un conjunto de datos completamente independiente y recogido en un marco temporal posterior.
- **Validación interna:** Procedimiento técnico utilizado durante la fase de entrenamiento para medir la estabilidad y el error estimado de un algoritmo empleando los datos del propio *dataset* original.
- **Vector de características:** Estructura numérica ordenada que agrupa los 25 atributos cuantitativos extraídos de una URL para alimentar los modelos y permitir su clasificación.
- **XGBoost (eXtreme Gradient Boosting):** Implementación altamente optimizada y eficiente del algoritmo de *Boosting* basada en árboles de decisión secuenciales.



## 7 Bibliografía

- [1] IBM. ¿Qué es el phishing?. 2024. Disponible en: <https://www.ibm.com/es-es/think/topics/phishing> (Consultado el 20 de marzo de 2026).
- [2] APWG. Phishing Activity Trends Report Q4 2025. 2025. Disponible en: [https://docs.apwg.org/reports/apwg\\_trends\\_report\\_q4\\_2025.pdf](https://docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf) (Consultado el 16 de marzo de 2026).
- [3] AEPD. 10 Malentendidos sobre el Machine Learning. Agencia Española de Protección de Datos, Madrid, 2022. Disponible en: <https://www.aepd.es/guias/10-malentendidos-machinelearning.pdf> (Consultado el 22 de marzo de 2026).
- [4] ISACA. Artificial Intelligence's Dual Role in Phishing Attacks. ISACA Journal, Volume 1, 2025. Disponible en: <https://www.isaca.org/resources/isaca-journal/issues/2025/volume-1/artificial-intelligences-dual-role-in-phishing-attacks> (Consultado el 3 de marzo de 2026).
- [5] SCHWARTZ, R., DODGE, J., SMITH, N. A., & ETZIONI, O. Green AI. Communications of the ACM, pp. 54-63, Vol. 63 / No. 12, 2020. Disponible en: <https://dl.acm.org/doi/10.1145/3381831> (Consultado el 23 de marzo de 2026).
- [6] ENISA. Threat Landscape. European Union Agency for Cybersecurity, 2025. Disponible en: <https://www.enisa.europa.eu/topics/cyber-threats/threat-landscape> (Consultado el 12 de marzo de 2026).
- [7] IBM. Spear Phishing vs. phishing: ¿cuál es la diferencia?. 2023. Disponible en: <https://www.ibm.com/es-es/think/topics/spear-phishing-vs-standard-phishing> (Consultado el 16 de marzo de 2026).
- [8] MERTENS, X. Add Punycode to your Threat Hunting Routine. SANS Internet Storm Center (ISC), 2026. Disponible en: <https://isc.sans.edu/diary/32640> (Consultado el 16 de marzo de 2026).
- [9] IBM. What is Machine Learning?. 2025. Disponible en: <https://www.ibm.com/think/topics/machine-learning> (Consultado el 14 de marzo de 2026).
- [10] IBM. ¿Qué son los algoritmos de Machine Learning?. 2025. Disponible en: <https://www.ibm.com/es-es/think/topics/machine-learning-algorithms> (Consultado el 22 de marzo de 2026).
- [11] BREIMAN, L. Random Forests. Machine Learning, pp. 5-32, Vol. 45 / No. 1, 2001. Disponible en: <https://doi.org/10.1023/A:1010933404324> (Consultado el 23 de marzo de 2026).

- [12] IBM. ¿Qué es Support Vector Machine?. 2023. Disponible en: <https://www.ibm.com/mx-es/think/topics/support-vector-machine> (Consultado el 23 de marzo de 2026).
- [13] IBM. ¿Qué es XGBoost?. 2024. Disponible en: <https://www.ibm.com/es-es/think/topics/xgboost> (Consultado el 23 de marzo de 2026).
- [14] DHANUSHKODI, K. & THEJAS, S. AI Enabled Threat Detection: Leveraging Artificial Intelligence for Advanced Security and Cyber Threat Mitigation. IEEE Access, Vol. 12, pp. 165682-165701, 2024. Disponible en: <https://ieeexplore.ieee.org/document/10747338> (Consultado el 24 de marzo de 2026).
- [15] PRASAD, A. & CHANDRA, S. PhiUSIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. Computers & Security, Vol. 136, 2024. Disponible en: <https://doi.org/10.1016/j.cose.2023.103545> (Consultado el 10 de marzo de 2026).
- [16] GOOGLE. Mitigating prompt injection attacks with a layered defense strategy. Google Security Blog, 2025. Disponible en: <https://blog.google/security/mitigating-prompt-injection-attacks/> (Consultado el 16 de marzo de 2026).
- [17] MICROSOFT. Scaling Security Operations with Microsoft Defender Autonomous Defense and Expert-led Services. Microsoft Security Blog, 2026. Disponible en: <https://www.microsoft.com/en-us/security/blog/2026/02/24/scaling-security-operations-with-microsoft-defender-autonomous-defense-and-expert-led-services/> (Consultado el 16 de marzo de 2026).
- [18] CLOUDFLARE. Email Security | Anti-Phishing Protection. 2025. Disponible en: <https://www.cloudflare.com/sase/products/email-security/> (Consultado el 16 de marzo de 2026).
- [19] FORRESTER RESEARCH. The Forrester Wave™: Email Security, Q2 2025. 2025. Disponible en: <https://www.cloudflare.com/lp/forrester-wave-email-security-2025/> (Consultado el 16 de marzo de 2026).
- [20] EBUBEKIRBBR. Phishing Dataset (pdd). GitHub Repository, 2018. Disponible en: <https://github.com/ebubekirbbr/pdd/tree/master/input> (Consultado el 10 de abril de 2026).
- [21] CISCO TALOS INTELLIGENCE GROUP. PhishTank: Join the fight against phishing. 2026. Disponible en: <https://www.phishtank.com> (Consultado el 20 de marzo de 2026).
- [22] DOMCOP. Top 10 Million Domains (Based on Open PageRank data). 2026. Disponible en: <https://www.domcop.com/top-10-million-domains> (Consultado el 12 de abril de 2026).

- [23] IBM. What is Model Drift?. 2024. Disponible en:  
<https://www.ibm.com/think/topics/model-drift> (Consultado el 10 de abril de 2026).
- [24] ZHANG, H. The Optimality of Naive Bayes. Proceedings of the Seventeenth International Florida Artificial Intelligence Research Society Conference (FLAIRS), 2004. Disponible en:  
<https://cdn.aaai.org/FLAIRS/2004/Flairs04-097.pdf> (Consultado el 6 de abril de 2026).
- [25] NG, A. Y. & JORDAN, M. I. On Discriminative vs. Generative Classifiers: A comparison of logistic regression and naive Bayes. Advances in Neural Information Processing Systems (NIPS), Vol. 14, 2001. Disponible en:  
<https://papers.nips.cc/paper/2001/file/7b7a53e239400a13bd6be6c91c4f6c4e-Paper.pdf> (Consultado el 6 de abril de 2026).
- [26] ALTMAN, N. S. An Introduction to Kernel and Nearest-Neighbor Nonparametric Regression. The American Statistician, Vol. 46 / No. 3, pp. 175-185, 1992. Disponible en:  
[https://sites.stat.washington.edu/courses/stat527/s13/readings/Altman\\_AmStat\\_1992.pdf](https://sites.stat.washington.edu/courses/stat527/s13/readings/Altman_AmStat_1992.pdf) (Consultado el 6 de abril de 2026).
- [27] QUINLAN, J. R. Induction of Decision Trees. Machine Learning, Vol. 1 / No. 1, pp. 81-106, 1986. Disponible en:  
<https://link.springer.com/article/10.1007/BF00116251> (Consultado el 6 de abril de 2026).
- [28] POPESCU, M. C., BALAS, V. E., PERESCU-POPESCU, L. & MASTORAKIS, N. E. Multilayer perceptron and neural networks. WSEAS Transactions on Circuits and Systems, Vol. 8 / No. 7, 2009. Disponible en:  
[https://www.researchgate.net/publication/228340819\\_Multilayer\\_perceptron\\_and\\_neural\\_networks](https://www.researchgate.net/publication/228340819_Multilayer_perceptron_and_neural_networks) (Consultado el 6 de abril de 2026).
- [29] SOKOLOVA, M. & LAPALME, G. A systematic analysis of performance measures for classification tasks. Information Processing & Management, Vol. 45 / No. 4, pp. 427-437, 2009. Disponible en:  
<https://www.sciencedirect.com/science/article/pii/S0306457309000259> (Consultado el 7 de abril de 2026).
- [30] FAWCETT, T. An Introduction to ROC analysis. Pattern Recognition Letters, Vol. 27 / No. 8, pp. 861-874, 2006. Disponible en:  
[https://www.researchgate.net/publication/222511520\\_Introduction\\_to\\_ROC\\_analysis](https://www.researchgate.net/publication/222511520_Introduction_to_ROC_analysis) (Consultado el 7 de abril de 2026).

## 8 Anexos

### **Anexo I: Repositorio de Código y Datos**

Todo el material desarrollado para este Trabajo de Fin de Grado, incluyendo los scripts de preprocesamiento, el estudio de obsolescencia, la evaluación de algoritmos y los datasets procesados, se encuentra disponible de forma pública en el siguiente repositorio de GitHub:

**URL:** <https://github.com/scano-0/Deteccion-Phishing-Machine-Learning>

El repositorio está organizado en las siguientes secciones:

- /notebooks: Código fuente en formato Jupyter Notebook.
- /data: Conjuntos de datos originales y procesados.
- README.md: Documentación técnica.